



Министерство науки и высшего образования Российской Федерации
Мытищинский филиал
Федерального государственного автономного образовательного
учреждения
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МФ МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Космический»

КАФЕДРА «Прикладная математика, информатика и вычислительная техника»

РАСЧЕТНО-ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
К ВЫПУСКНОЙ КВАЛИФИКАЦИОННОЙ РАБОТЕ
НА ТЕМУ:

Разработка нейросетевой модели для поддержки
принятия решения при маршрутизации пожарной
автотехники в городской среде

Студент КЗ-85Б

(Подпись, дата)

Р. А. Нистратов
(И.О.Фамилия)

Руководитель ВКР

(Подпись, дата)

В.Я. Вилисов
(И.О.Фамилия)

Нормоконтролер

(Подпись, дата)

А.В. Чернышов
(И.О.Фамилия)

2026 г.

АННОТАЦИЯ

Выпускная квалификационная работа содержит 104 страниц текстового документа, 16 рисунков, 2 таблицы, 16 использованных источников.

Ключевые слова: ИМИТАЦИОННОЕ МОДЕЛИРОВАНИЕ, ГЛУБОКОЕ ОБУЧЕНИЕ С ПОДКРЕПЛЕНИЕМ, DQN, МАРШРУТИЗАЦИЯ, ЭКСТРЕННЫЕ СЛУЖБЫ, ANYLOGIC, ИСКУССТВЕННЫЙ ИНТЕЛЛЕКТ, СППР.

Объектом исследования являются системы поддержки принятия решений (СППР) для оперативного реагирования служб экстренного вызова в динамической городской среде.

Цель работы — разработка нейросетевой модели и комплекса программных средств для повышения эффективности маршрутизации гетерогенной пожарной автотехники в условиях города, включая создание двухуровневых интерфейсов взаимодействия с пользователями.

В процессе выполнения работы проведен системный анализ существующих подходов к диспетчеризации пожарных частей. Выявлена критическая уязвимость классических детерминированных (жадных) алгоритмов, приводящая к эффекту «ресурсного голода» при распределении дефицитной спецтехники (автолестниц) в условиях высокой плотности инцидентов.

Для решения данной проблемы спроектирована и программно реализована распределенная архитектура, объединяющая высокодетализированную среду агентного моделирования AnyLogic [2] и вычислительное ядро на языке Python (фреймворк PyTorch) [15]. Надежная синхронизация компонентов в режиме реального времени обеспечена за счет внедрения бестаймаутного сокетного шлюза Pipeline [14].

На базе Марковского процесса принятия решений (MDP) [1, 8] разработана и обучена модель глубокой Q-сети (DQN) [11]. Нейросетевой агент реализует стратегию предиктивного резервирования спецтехники, опираясь на эвристические матрицы дистанций и актуальные статусы пожарных депо.

Дополнительно спроектированы эргономичные интерфейсы: рабочее место оперативного диспетчера с динамической ГИС-картой и панель администратора для управления гиперпараметрами обучения.

Экспериментальное исследование (А/В тестирование) в условиях имитации пиковой стрессовой нагрузки доказало эффективность разработанной СППР. Применение нейросетевой модели позволило сократить среднее время реагирования до 62% (относительно базового алгоритма) и уменьшить долю нарушений регламента согласно Федеральному закону № 123-ФЗ [9].

СОДЕРЖАНИЕ

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ.....	6
СПИСОК СОКРАЩЕНИЙ	8
ВВЕДЕНИЕ.....	10
1 ОЩЕТЕХНИЧЕСКОЕ ОБОСНОВАНИЕ.....	14
1.1 Основные задачи и требования к системам маршрутизации в пожарной службе.....	14
1.2 Критерии эффективности и ограничения.....	15
1.3 Анализ существующих методов поддержки принятия решений	16
1.4 Выводы	18
2 РЕШЕНИЕ ЗАДАЧИ.....	19
2.1 Разработка архитектуры нейросетевой модели и описание среды задачи.....	19
2.1.1 Системный анализ и многокомпонентная архитектура программного комплекса СППР	19
2.1.2 Протокол сопряжения и синхронизации имитационного и нейросетевого времени.....	21
2.1.3 Формализация и декомпозиция пространственной среды задачи	23
2.1.4 Математическое описание потока инцидентов и структуры гетерогенных ресурсов	25
2.2 Выбор типа и структуры нейросети, описание процесса сбора и обработки данных	28
2.2.1 Агрегация и векторизация состояния	28
2.2.2 Архитектура глубокой Q-сети (DQN)	29
2.2.3 Политика выбора действий и расчет функции вознаграждения ...	30
2.2.4 Механизм буфера памяти (Replay Buffer) и алгоритм обучения...	32

2.3 Разработка пользовательских интерфейсов	36
2.3.1 Интерфейс для лица, принимающего решение (оперативного диспетчера)	36
2.3.2 Интерфейс службы поддержки и администрирования модели.....	39
2.4 Реализация и обучение нейросетевой модели на тестовых данных	41
2.5 Критерии оценки качества маршрутизации и интерфейсных решений	43
2.6 Методология, условия проведения экспериментов и программный комплекс	44
2.7 Сценарии моделирования.....	47
2.8 Описание сравниваемых алгоритмов и принцип обучения нейросети	48
2.9 Анализ результатов вычислительных экспериментов.....	52
2.9.1 Анализ Сценария № 1 (Низкая нагрузка)	52
2.9.2 Анализ Сценария № 2 (Высокая нагрузка)	55
2.9.3 Обобщенный сравнительный анализ эффективности и стабильности.....	57
2.10 Выводы.....	61
ЗАКЛЮЧЕНИЕ	62
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	66
ПРИЛОЖЕНИЕ А	68
ПРИЛОЖЕНИЕ Б.....	82

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ

В настоящей выпускной квалификационной работе применяются следующие термины с соответствующими определениями:

Агент — автономная программная сущность в имитационной модели или алгоритме искусственного интеллекта, способная воспринимать состояние окружающей среды и совершать целенаправленные действия для максимизации заданного критерия эффективности. Гетерогенный автопарк — совокупность транспортных средств оперативных служб, состоящая из единиц техники различного целевого (тактического) назначения, обладающих различными массогабаритными и динамическими характеристиками (в рамках данного исследования — пожарные автоцистерны и автолестницы). Глубокое обучение с подкреплением (Deep Reinforcement Learning) — раздел машинного обучения, в котором агент обучается оптимальному поведению в интерактивной среде методом проб и ошибок, используя глубокие искусственные нейронные сети для аппроксимации функции ценности действий. Имитационное моделирование — метод исследования, при котором изучаемая физическая система заменяется компьютерной моделью, с достаточной точностью описывающей реальные процессы, с целью проведения контролируемых вычислительных экспериментов. Марковский процесс принятия решений — математическая формализация для задач последовательного принятия решений в условиях стохастической среды, базирующаяся на свойстве отсутствия последействия (вероятность перехода в новое состояние зависит исключительно от текущего состояния среды и выбранного действия). Состояние среды (State) — формализованный многомерный вектор данных, описывающий текущую ситуацию в имитационной модели в конкретный момент времени (включает координаты инцидента, матрицы дистанций и статус доступности техники в депо). Функция вознаграждения / штрафа (Reward Function) — математическая функция, генерирующая скалярный сигнал отклика виртуальной среды на действия агента, используемая для корректировки весов нейронной сети в процессе градиентного спуска. Эпоха (Эпизод) обучения — один изолированный

одношаговый цикл взаимодействия нейросетевого агента с имитационной средой в рамках обслуживания одного инцидента. Эпизод начинается в момент генерации вызова, включает фиксацию текущего состояния системы и выбор действия агентом, и принудительно завершается (переходит в терминальное поглощающее состояние) в момент фактического прибытия целевой техники к очагу с генерацией итоговой награды (штрафа).

СПИСОК СОКРАЩЕНИЙ

В настоящей выпускной квалификационной работе применяются следующие сокращения и обозначения:

АЛ — автолестница (специальный пожарный автомобиль).

АЦ — автоцистерна (основной пожарный автомобиль).

БД — база данных.

ВКР — выпускная квалификационная работа.

ГИС — геоинформационная система.

ДТП — дорожно-транспортное происшествие.

ИИ — искусственный интеллект.

МЧС — Министерство по делам гражданской обороны, чрезвычайным ситуациям и ликвидации последствий стихийных бедствий.

ПЭВМ — персональная электронно-вычислительная машина.

СППР — система поддержки принятия решений.

УДС — улично-дорожная сеть.

ФПС ГПС — Федеральная противопожарная служба Государственной противопожарной службы.

ABM — Agent-Based Modeling (агентное моделирование).

API — Application Programming Interface (программный интерфейс приложения).

DQN — Deep Q-Network (глубокая Q-сеть).

DRL — Deep Reinforcement Learning (глубокое обучение с подкреплением).

GPU — Graphics Processing Unit (графический процессор).

HMI — Human-Machine Interface (человеко-машинный интерфейс).

JSON — JavaScript Object Notation (текстовый формат обмена данными).

JVM — Java Virtual Machine (виртуальная машина Java).

MDP — Markov Decision Process (марковский процесс принятия решений).

MSELoss — Mean Squared Error Loss (функция потерь среднеквадратичной ошибки).

MVP — Minimum Viable Product (минимально жизнеспособный продукт).

OSM — OpenStreetMap (свободная географическая карта мира).

ReLU — Rectified Linear Unit (функция активации «выпрямленный линейный блок»).

TCP/IP — Transmission Control Protocol/Internet Protocol (протокол управления передачей/межсетевой протокол).

UI/UX — User Interface / User Experience (пользовательский интерфейс / пользовательский опыт).

ВВЕДЕНИЕ

Актуальность темы исследования. На современном этапе развития мегаполисов и крупных городских агломераций обеспечение комплексной безопасности населения и инфраструктуры является одной из приоритетных задач муниципальных и государственных служб. Среди всех видов техногенных и природных угроз пожары в городской черте представляют наибольшую опасность с точки зрения потенциальных человеческих жертв, экономического ущерба и разрушения объектов критической инфраструктуры. Эффективность ликвидации последствий чрезвычайных ситуаций техногенного характера, связанных с возгораниями, напрямую зависит от временных показателей реагирования подразделений пожарной охраны. Согласно нормативным требованиям, время прибытия первых пожарных расчетов к месту вызова в городских поселениях не должно превышать установленных регламентированных лимитов.

Однако динамическое развитие современной городской среды накладывает существенные ограничения на процессы оперативного реагирования. Ключевыми дестабилизирующими факторами выступают перманентный рост плотности застройки, усложнение геометрии дорожно-транспортных сетей и прогрессирующий дефицит пропускной способности улично-дорожной сети, выражающийся в образовании масштабных заторов (пробок). Традиционные методы диспетчеризации и маршрутизации, применяемые в автоматизированных системах экстренных служб, преимущественно опираются на статические алгоритмы поиска кратчайшего пути (такие как алгоритм Дейкстры или A^*) на фиксированных графах дорожной сети [5, 6]. Подобные подходы демонстрируют критическое снижение эффективности в условиях динамически изменяющегося трафика, поскольку они не способны прогнозировать нелинейное развитие дорожной ситуации и учитывать пространственно-временные заторы в режиме реального времени.

Дополнительная сложность оперативного управления заключается в гетерогенности (разнородности) парка пожарной автотехники и специфике ее

применения. Для успешного подавления очага возгорания и спасения людей зачастую требуется одновременное или строго скоординированное привлечение различных типов спецтехники, обладающих существенно различающимися массогабаритными и динамическими характеристиками — в частности, основных пожарных автомобилей (автоцистерн) и специальных пожарных автомобилей (автолестниц). Автолестницы, имея значительно большие габариты и меньшую маневренность, в гораздо большей степени подвержены задержкам в дорожных заторах, при этом их дефицит на месте происшествия при пожарах в высотных зданиях является критическим. В условиях ограниченности ресурсов пожарных депо и высокой интенсивности поступающих заявок задача распределения и маршрутизации техники превращается в многокритериальную задачу стохастической оптимизации.

В этой связи возникает объективная необходимость в разработке качественно новых интеллектуальных систем поддержки принятия решений (СППР) для оперативного диспетчера пожарной охраны. Наиболее перспективным направлением решения указанной проблемы является применение методов искусственного интеллекта, а именно глубокого обучения с подкреплением (Deep Reinforcement Learning, DRL), интегрированного со средами агентного имитационного моделирования [7, 8]. Имитационное моделирование позволяет воссоздать высокоточную цифровую копию городской инфраструктуры, включая ГИС-карты, логику возникновения инцидентов и динамику транспортных потоков [2, 4]. В свою очередь, нейросетевая модель, обучающаяся в такой интерактивной среде, способна аккумулировать скрытые закономерности, адаптироваться к топологии конкретного города, учитывать текущую дислокацию и доступность разнородной техники, а также нелинейно реагировать на появление дорожных пробок. Таким образом, разработка нейросетевых моделей для оптимизации управления и маршрутизации экстренных служб является актуальной научно-технической задачей, находящейся на стыке системного анализа, теории искусственного интеллекта и имитационного моделирования.

Объект исследования: Системы поддержки принятия решений для оперативного реагирования служб экстренного вызова (на примере пожарной автотехники) в городской среде.

Предмет исследования: Методы, алгоритмы и интерфейсные решения построения, обучения и применения нейросетевых моделей для оптимизации маршрутизации пожарной техники с учетом динамических городских условий и пользовательских сценариев взаимодействия.

Цель исследования: Разработка нейросетевой модели и программных средств для повышения эффективности маршрутизации пожарной автотехники в условиях города, включая создание интерфейсов взаимодействия с пользователями различных уровней.

Для достижения поставленной цели необходимо решить следующие **задачи исследования:**

Анализ существующих систем автоматизированной маршрутизации пожарной техники, принципов функционирования экстренных служб и ключевых факторов, влияющих на процесс принятия решений диспетчером.

Анализ современных нейросетевых методов, алгоритмов глубокого обучения с подкреплением (RL), применяемых для маршрутизации транспорта и автоматизации задач экстренного реагирования.

Разработка математической постановки задачи в виде марковского процесса принятия решений (MDP) и проектирование архитектуры нейросетевой СППР.

Разработка комплексной имитационной модели городской среды на базе ГИС-карты в среде AnyLogic, включающей стохастическую генерацию дорожных заторов и инцидентов различного уровня сложности.

Проектирование и программная реализация двустороннего интерфейса интеграции имитационной среды и нейросетевого модуля на языке Python с использованием специализированного шлюза Pipeline.

Разработка пользовательских интерфейсов для оперативного диспетчера и администратора системы с возможностью динамического переключения режимов управления.

Проведение экспериментального исследования, анализ сходимости алгоритма обучения нейросети и сравнительная оценка эффективности предложенного решения по отношению к базовым детерминированным алгоритмам диспетчеризации.

Методы исследования. Для решения поставленных задач использовались методы системного анализа, теории марковских процессов, теории систем с подкреплением (Reinforcement Learning) и глубоких Q-сетей (DQN), методы объектно-ориентированного программирования, имитационного и агентного моделирования, а также методы проектирования пользовательских интерфейсов (UI/UX).

Научная новизна работы заключается в разработке комплексного подхода к маршрутизации экстренных служб, совмещающего агентное имитационное моделирование городской среды с нейросетевым алгоритмом глубокого обучения с подкреплением, который учитывает динамический дефицит разнородных типов техники (автоцистерн и автолестниц) и нелинейное влияние дорожных заторов в режиме реального времени.

1 ОЩЕТЕХНИЧЕСКОЕ ОБОСНОВАНИЕ

1.1 Основные задачи и требования к системам маршрутизации в пожарной службе

В данной части проводится системный анализ предметной области, включающий специфику оперативного реагирования пожарно-спасательных подразделений в условиях современной городской среды. Рассматриваются существующие алгоритмы диспетчеризации и маршрутизации спецтехники, а также выявляются ключевые ограничения классических детерминированных (жадных) подходов при работе с гетерогенным и дефицитным автопарком. На основе проведенного исследования формулируется постановка задачи и обосновывается выбор методов имитационного моделирования и глубокого обучения с подкреплением в качестве оптимального стека технологий для создания системы поддержки принятия решений.

Управление силами и средствами пожарной охраны на урбанизированных территориях подчинено жестким нормативно-правовым и тактико-техническим требованиям. Согласно Федеральному закону № 123-ФЗ «Технический регламент о требованиях пожарной безопасности», ключевым интегральным критерием успешности функционирования оперативных служб является время прибытия первого пожарного подразделения к месту вызова, которое в черте городских поселений не должно превышать 10 минут [9]. В условиях реального мегаполиса задача обеспечения этого норматива декомпозируется на ряд специфических оперативных задач, возлагаемых на автоматизированные системы диспетчеризации и маршрутизации [3]. К основным задачам относятся: Автоматическая верификация адреса и определение ранга (сложности) пожара на основе первичного сообщения. Идентификация доступных сил и средств в ближайших пожарно-спасательных гарнизонах. Принятие решения о назначении конкретных боевых расчетов из различных пожарных депо (задача системной диспетчеризации). Расчет оптимальной траектории следования назначенной техники к месту инцидента с учетом текущих дорожных ограничений (задача

гео-маршрутизации). Главное системное требование к СППР диспетчера заключается в необходимости учета гетерогенности (разнородности) и дефицитности оперативного автопарка. Реальная структура автопарков экстренных служб устроена значительно сложнее и включает в себя обширный перечень узкоспециализированной техники. Тем не менее, для изоляции исследуемых системных эффектов и снижения вычислительной сложности в работе принято авторское распределение парка по двум ключевым типам. Первый тип — это универсальные автоцистерны (АЦ), составляющие около 80% парка и обеспечивающие доставку личного состава и огнетушащих веществ. Второй тип — дефицитные высокогабаритные автолестницы (АЛ), составляющие около 20% парка и необходимые для спасательных работ на высоте. Введение данной абстракции позволяет перевести задачу диспетчеризации из плоскости стандартного поиска кратчайшего пути для абстрактной машины в плоскость координационного распределения разнородных ресурсов. Система оптимизирует назначение редкого компонента (АЛ) так, чтобы минимизировать интегральный риск для города и не допускать «вымывания» спецтехники из критических зон.

1.2 Критерии эффективности и ограничения

Процесс оптимизации управления пожарными расчетами протекает в условиях жестких инфраструктурных и средовых ограничений. Традиционно ключевым критерием эффективности автоматизированных систем логистики считается минимизация пройденного расстояния ($S \rightarrow \min$). Однако для экстренных служб данный критерий является вторичным, уступая приоритет критерию минимизации интегрального времени реагирования ($T_{\text{реag}} \rightarrow \min$), а также критерию надежности доставки целевого типа техники.

Ключевыми дестабилизирующими факторами и ограничениями городской среды выступают: Стохастическая динамика дорожного графа (пробки): Суточные колебания трафика, заторы и ДТП нелинейно изменяют пропускную способность дорог, увеличивая время следования спецтехники в разы.

Габаритные и динамические ограничения техники: Пожарные автолестницы (АЛ) обладают значительно большей массой, сниженной маневренностью и увеличенным радиусом разворота по сравнению с автоцистернами. Участки дорог, условно доступные для объезда пробок легковым транспортом или легкими АЦ (узкие дворовые проезды, межквартальные дублеры), часто физически заблокированы для проезда тяжелых АЛ. Системный дефицит ресурсов: Стохастический характер возникновения пожаров создает ситуацию, когда несколько инцидентов могут произойти в смежных районах одновременно. Важнейшее ограничение существующих ГИС: Современные общегражданские навигационные системы (например, Яндекс.Навигатор, Google Maps, открытые движки OSRM) идеально решают задачу изолированной гео-маршрутизации (точечного обхода пробок). Они способны эффективно проложить путь для конкретной машины из пункта А в пункт Б по текущему дорожному графу. Однако эти системы абсолютно слепы к управлению дефицитом ресурсов экстренных служб. Они функционируют на основе «жадной» логики: если диспетчер запрашивает машину, навигатор просто укажет на ближайшее депо. Гражданский навигатор не способен спрогнозировать, что посылка последней доступной в секторе автолестницы на мелкий вызов приведет к системной катастрофе (нехватке АЛ), если через три минуты в этом же секторе загорится высотное здание. Таким образом, ограничением является не отсутствие алгоритмов объезда пробок, а отсутствие в классических ГИС механизмов долгосрочного планирования и прогнозирования рисков распределения гетерогенного парка техники.

1.3 Анализ существующих методов поддержки принятия решений

Для автоматизации процессов диспетчеризации и управления транспортными потоками в рамках СППР исторически применяются два больших класса методов: классические детерминированные алгоритмы и современные интеллектуальные подходы на базе нейросетевых моделей.

Классические детерминированные подходы

К ним относятся алгоритмы поиска кратчайших путей на графах (алгоритм Дейкстры, A^* , алгоритм Флойда-Уоршелла), а также методы математического программирования для решения задач маршрутизации транспорта (Vehicle Routing Problem, VRP). Алгоритм Дейкстры и алгоритм A^* находят точное оптимальное решение на статическом или квазидинамическом графе за полиномиальное время [6]. Их преимущество — математическая строгость и гарантированная сходимость. Недостатки: данные методы локализованы в рамках одной сессии расчета. Они рассчитывают маршрут «здесь и сейчас». При высокой плотности заторов и гетерогенном автопарке классические методы не могут увязать воедино текущую дислокацию машин, топологию пробок и ранг пожара для принятия стратегического решения о том, из какого именно депо (даже если оно географически дальше) выгоднее отправить дефицитную лестницу.

Нейросетевые методы

Нейросетевые методы предлагают перенос центра тяжести вычислительной нагрузки с оперативного расчета путей на предварительное обучение интеллектуального агента. В задачах управления экстренными службами наибольшую эффективность демонстрирует аппарат DRL, в частности, алгоритмы DQN [11]. В рамках этой концепции городская среда (включая пробки и состояния пожарных частей) формализуется как Марковский процесс принятия решений (MDP) [1, 8]: Вектор состояний (*State*): включает в себя не только пространственные матрицы дистанций до пожара, но и матрицы доступности техники по типам (количество свободных АЦ и АЛ в каждой пожарной части). Пространство действий (*Action*): выбор конкретной пожарной станции или комбинации станций для отправки экипажей на верифицированный инцидент. Функция вознаграждения (*Reward*): формируется как штраф за суммарное время прибытия, дополненный жесткими отрицательными штрафами за системные ошибки (например, отправка АЦ вместо критически необходимой АЛ, или полное исчерпание резерва АЛ в высотном районе). Преимущество нейросетевого DQN-агента заключается в его способности аккумулировать

латентные пространственно-временные закономерности города в процессе обучения в имитационной среде. Обученная нейросеть за доли миллисекунды способна оценить скрытые риски и выдать диспетчеру рекомендацию, превосходящую «жадный» алгоритм. ИИ учится стратегически «придерживать» дефицитные автолестницы в частях, граничащих с высотными микрорайонами, перенаправляя на простые вызовы технику из удаленных зон, если это не нарушает критический временной порог.

1.4 Выводы

Управление пожарными расчетами в мегаполисе регламентировано жестким 10-минутным лимитом времени прибытия, достижение которого осложнено динамическими заторами и гетерогенной структурой автопарка (дефицит спецтехники АЛ составляет около 20% от общего объема сил). Существующие ГИС и коммерческие навигационные системы эффективно решают локальную задачу гео-маршрутизации (прокладка пути с объездом пробок), но принципиально неприменимы для решения задач глобальной диспетчеризации экстренных служб, так как они не учитывают риски долгосрочного дефицита гетерогенных ресурсов в масштабах города. Классические алгоритмы на графах (A^* , Дейкстры) используют «жадную» логику выбора ближайшего депо и не способны адаптироваться к комплексным стохастическим изменениям среды. Наиболее перспективным решением является разработка СППР на основе нейросетевых моделей глубокого Q-обучения (DQN), интегрированных со средами агентного моделирования. Такой подход позволяет ИИ выступать в роли макро-координатора, оптимизирующего распределение редких типов спецтехники с опорой на долгосрочную минимизацию операционных рисков и прогнозируемое время задержек в пути.

2 РЕШЕНИЕ ЗАДАЧИ

2.1 Разработка архитектуры нейросетевой модели и описание среды задачи

2.1.1 Системный анализ и многокомпонентная архитектура программного комплекса СППР

Данная часть посвящена практической реализации разработанной нейросетевой модели и программного комплекса в целом. В ней подробно описывается структурно-функциональная архитектура системы, процесс построения имитационной ГИС-модели в среде AnyLogic, а также логика генерации стохастического потока инцидентов. Особое внимание уделено алгоритмам интеграции симуляционной среды с вычислительным модулем на языке Python, математическому аппарату обучения DQN-агента, а также проектированию эргономичных пользовательских интерфейсов для оперативного диспетчера и администратора.

Разработка систем поддержки принятия решений (СППР) для координации экстренных ведомств в крупных городах сталкивается с фундаментальным вычислительным конфликтом между необходимостью высокой детализации симуляционной среды и требованиями к производительности машинного обучения. С одной стороны, для достоверного прогнозирования времени прибытия гетерогенных пожарных расчетов критически важно микроскопическое агентное моделирование (Agent-Based Modeling, ABM) [2]. Этот подход позволяет воссоздать сложную городскую логику: геометрию дорожных графов, влияние габаритов спецтехники и стохастическую природу возникновения заторов в часы пик. С другой стороны, эффективное тренировка агентов методами глубокого обучения с подкреплением (DRL) требует десятков тысяч итераций и высокоскоростной обработки многомерных тензоров, что крайне неэффективно внутри стандартных движков имитационного моделирования. В настоящей работе данное противоречие снимается за счет проектирования двухуровневой распределенной архитектуры. Она физически

разграничивает тяжелый контур имитации пространственных процессов и специализированное вычислительное ядро нейросети, обеспечивая их строгую временную синхронизацию, отказоустойчивость и стабильную работу комплекса (Рисунок 2.1).



Рисунок 2.1 — Структурно-функциональная архитектура интегрированного комплекса AnyLogic – Pipeline – Python

Архитектура состоит из трех базовых уровней, каждый из которых выполняет строго изолированные системные функции:

Исполнительный уровень (Имитационная среда AnyLogic): Функционирует под управлением виртуальной машины Java (JVM) [4, 10]. Данный уровень берет на себя функции микроскопического моделирования физических процессов: инициализацию популяций гетерогенных транспортных агентов, расчет путей следования по графу дорожной сети, анимацию движения и фиксацию телеметрии.

Интеграционный уровень (Коммуникационный шлюз Pipeline): Выступает в роли сокет-сервера, обеспечивающего межпроцессное взаимодействие. Шлюз Pipeline не накладывает жестких ограничений на таймауты ожидания ответа, что критически важно для фазы стратегического планирования на стороне ИИ.

Уровень интеллектуального ядра (Вычислительная среда Python/PyTorch): Реализован на языке Python с использованием библиотек машинного обучения [13]. Модуль изолирован от графической оболочки

симулятора и сфокусирован на аппроксимации функции ценности действий на основе поступающих векторов состояний.

2.1.2 Протокол сопряжения и синхронизации имитационного и нейросетевого времени

Разделение программного комплекса на два изолированных исполнительных контура — имитационную среду AnyLogic (функционирующую в виртуальной машине Java — JVM) и вычислительное ядро на языке Python (фреймворк PyTorch) — продиктовано требованиями к производительности и вычислительной архитектуре систем глубокого обучения с подкреплением (DRL). Прямая реализация итерационных алгоритмов градиентного спуска и обработка многомерных тензоров внутри JVM сопряжены со значительными накладными расходами памяти и отсутствием нативной поддержки аппаратного ускорения (GPU Tensor Cores).

Для реализации бесшовной интеграции и преодоления межплатформенного барьера был спроектирован двусторонний событийно-блокирующий протокол на базе сокетного шлюза Pipeline. Межпроцессное взаимодействие (IPC — Inter-Process Communication) развернуто на локальном сетевом интерфейсе (loopback) с использованием транспортного протокола TCP/IP. Сетевой сокет гарантирует надежную доставку пакетов без искажения структуры данных.

Фундаментальной инженерной задачей при проектировании шлюза являлось обеспечение строгой инвариантности виртуального времени имитационной модели относительно времени выполнения вычислений нейросетью. Время в AnyLogic течет непрерывно и стохастически. Если в процессе генерации инцидента вычислительный контур Python затратит на выполнение прямого прохода нейросети (inference) 15 миллисекунд, в несинхронизированной системе виртуальное время AnyLogic продолжало бы идти вперед, что привело бы к искусственной задержке выезда пожарных расчетов на эквивалентное симуляционное время (которое может исчисляться минутами).

Выполнение системного времени AnyLogic принудительно приостанавливается с помощью вызова `getEngine().pause();`. Агент-диспетчер AnyLogic формирует вектор состояния S_t содержащий информацию о ранге пожара, координатах очага и доступности гетерогенной техники на станциях. Вектор сериализуется в JSON (Листинг 2.1), и передается через TCP/IP порт Pipeline в среду Python.

```
{
  "fire_id": 1042,
  "severity": 1,
  "distances": (42, 127, 70, 321),
  "station_status": [
    [2, 1],
    [0, 0],
    [2, 1],
    [1, 0]
  ]
}
```

Листинг 2.1 — Структура пакета вектора состояния S_t

Вычислительный поток Python десериализует пакет в тензор PyTorch, осуществляет прямое прохождение через слои глубокой Q-сети и выбирает оптимальное действие A_t (номер целевой пожарной части). Действие инкапсулируется в обратный JSON-ответ (Листинг 2.2) и передается в AnyLogic. AnyLogic принимает команду, распределяет вызов и возобновляет симуляцию: `getEngine().resume();`

```
{
  "fire_id": 1042,
  "selected_action": 1
}
```

Листинг 2.2 — Структура пакета команды действия A_t

2.1.3 Формализация и декомпозиция пространственной среды задачи

Пространственная структура имитационной модели развернута на базе реальных топографических данных OpenStreetMap (OSM) [12]. Внутри ГИС-пространства среды AnyLogic [5, 16] импортированная дорожная сеть автоматически преобразуется в сложный ориентированный взвешенный граф. Вершинами данного графа выступают перекрестки, развязки и тупики, а ребрами — участки улично-дорожной сети, по которым осуществляется непосредственная физическая навигация пожарной техники.

В этом виртуальном пространстве жестко зафиксированы статические координаты четырёх независимых пожарных депо, которые выступают стартовыми точками дислокации гетерогенного автопарка и формируют зоны первичной территориальной ответственности. Полная графическая визуализация разработанного ГИС-пространства, отражающая взаимное расположение базовых станций и структуру наложенного дорожного графа изображена на рисунке 2.2.

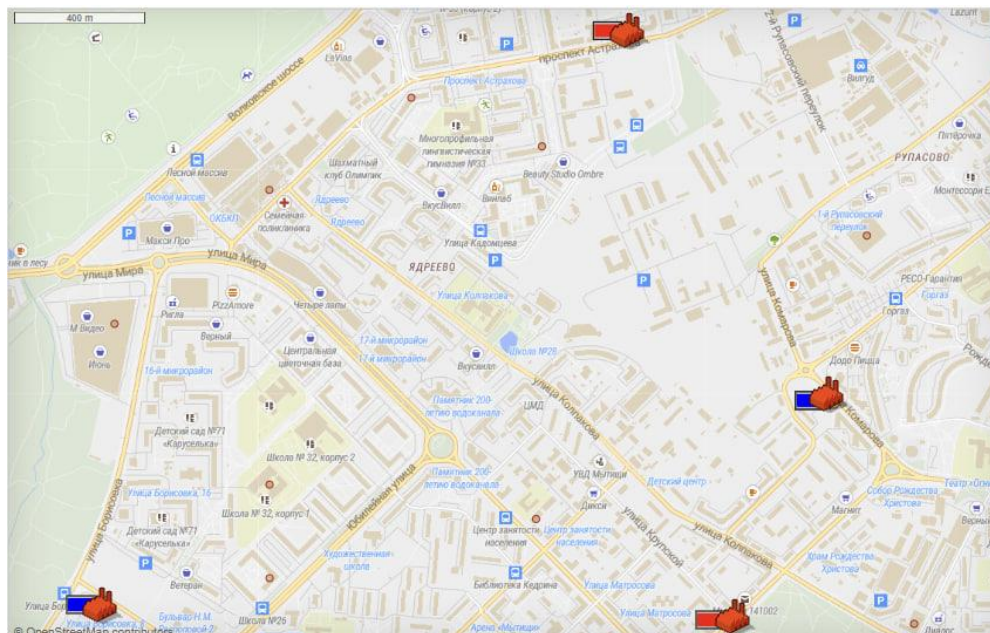


Рисунок 2.2 — Визуализация ГИС-среды моделирования в AnyLogic

Особое внимание при проектировании виртуальной среды было уделено созданию архитектурного задела для будущего масштабирования и усложнения модели. В частности, для графической и структурной имитации динамических сопротивлений дорожной сети в среду внедрены специализированные

полигональные объекты класса `RestrictionZone`, поименованные в коде как `jamZone`. Данные зоны предназначены для имитации участков скопления транспорта (пробок), где скорость движения автомобилей должна снижаться. Наглядная схема расположения данных полигонов загруженности и их пространственное наложение на транспортную систему мегаполиса также детально представлены на рисунке 2.3.

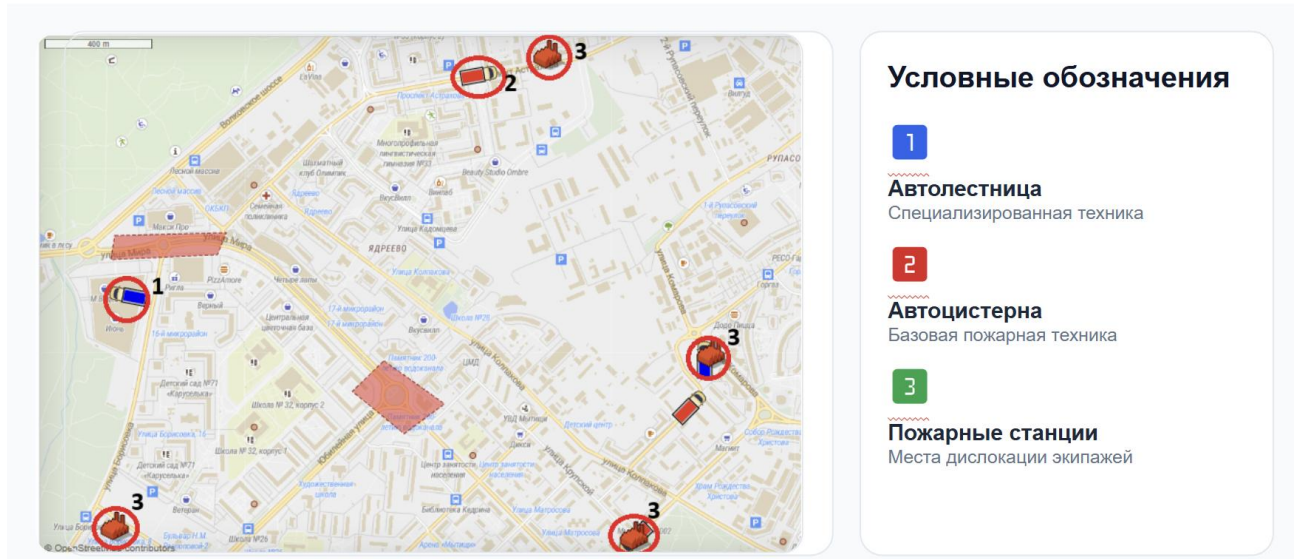


Рисунок 2.3 — Визуализация объектов на ГИС-карте среды моделирования в AnyLogic

Однако в рамках текущей программной реализации интеллектуального комплекса принято решение о семантическом разделении задач навигации и диспетчеризации. В настоящий момент данные о координатах, геометрии и плотности зон `jamZone` намеренно не передаются в вектор состояния S_t для нейронной сети. Нейросетевой агент функционирует в условиях частичной наблюдаемости (Partial Observability) относительно текущего городского трафика.

Такое архитектурное упрощение пространства состояний является осознанным: на данном этапе исследования фокус машинного обучения смещен исключительно на решение критической системной проблемы — ликвидацию структурного дефицита гетерогенных ресурсов (координация распределения универсальных автоцистерн и редких автолестниц). В текущей парадигме ИИ-агент «не видит» пробок напрямую, ориентируясь на эвристические матрицы

дистанций до очагов возгорания и статусы готовности пожарных частей. Присутствующие в модели объекты jamZone закладывают фундаментальную базу для последующих модификаций системы. В перспективе развития проекта планируется расширить размерность вектора состояний, передавая алгоритму DQN карту загруженности дорог в реальном времени, что позволит комплексу решать обе задачи одновременно: предиктивно управлять дефицитом спецтехники и осуществлять микроскопический объезд заторов.

2.1.4 Математическое описание потока инцидентов и структуры гетерогенных ресурсов

Процесс генерации чрезвычайных ситуаций (инцидентов) формализован в разработанной имитационной среде как пространственно-распределенный стохастический процесс с фиксированным шагом квантования времени [2, 4]. Логика возникновения потенциальных очагов возгорания привязана к фиксированному множеству пространственных ГИС-объектов, объединенных в системную коллекцию целевых точек городской инфраструктуры. Каждую единицу модельного времени (интервал в 1 минуту) элемент управления модели инициирует последовательный итерационный обход (цикл) по всем элементам данной ГИС-коллекции. Для каждого отдельного ГИС-узла независимо реализуется классическая схема испытаний Бернулли [4]. Вероятность возникновения инцидента в текущем узле на данном временном такте определяется параметром p ($0 \leq p \leq 1$), рассчитываемым встроенным генератором псевдослучайных чисел среды AnyLogic [16]. При фиксации положительного исхода испытания в конкретной точке, алгоритм симуляции производит стохастическое определение ранга (уровня) сложности ($Rank \in \{1,2\}$) инцидента на основе дискретного закона распределения вероятностей и формирует транзакт вызова для СППР.

В соответствии с руководящими документами МЧС России, в частности, «Порядком организации службы в подразделениях пожарной охраны» и Федеральным законом № 123-ФЗ [9], управление силами и средствами гарнизона

базируется на принципах жесткого регламентирования состава высланных сил в зависимости от присвоенного ранга (номера) пожара. Ранг пожара является основным оперативным признаком, определяющим масштаб чрезвычайной ситуации, потенциальную угрозу жизни людей и требуемое количество сил и средств для локализации и ликвидации возгорания. В рамках разработанной имитационной модели была произведена дискретизация реального многоуровневого расписания выездов до двух базовых операционных рангов сложности: $Rank = 1$ и $Rank = 2$. Данное упрощение является научно обоснованным декомпозиционным шагом, позволяющим изолировать исследуемую проблему дефицита гетерогенных ресурсов без потери общности тактической логики. Инциденты первого ранга сложности ($Rank = 1$). К данному классу относятся рутинные локальные возгорания на объектах с минимальным фактором высотности и низкой плотностью постоянного пребывания людей (например, одиночные транспортные средства, контейнерные площадки, изолированные складские постройки малой площади). Тактическая схема реагирования на инциденты $Rank = 1$ предполагает привлечение исключительно основных пожарных автомобилей — автоцистерн (TANKER), основной задачей которых является оперативная доставка боевого расчета и подача первичных огнетушащих веществ. В этих условиях привлечение специальной техники не требуется, а задача диспетчеризации носит детерминированный характер минимизации физического расстояния. Инциденты второго ранга сложности ($Rank = 2$). Данный класс моделирует сложные, динамически развивающиеся пожары на критических объектах городской инфраструктуры: многоэтажных жилых комплексах, административных зданиях, высотных бизнес-центрах. Тактика тушения таких пожаров накладывает жесткое пространственно-временное ограничение: успешное развертывание сил возможно только при условии одновременного или строго скоординированного прибытия как универсальных автоцистерн (TANKER), так и автолестниц (LADDER). Автолестницы в данном контексте выступают незаменимым фактором спасения, обеспечивая пути эвакуации

личного состава и гражданских лиц с верхних ярусов зданий, а также возможность подачи лафетных стволов на значительную высоту. Следовательно, при возникновении вызова класса $Rank = 2$ интегральное время успешного развертывания боевого участка лимитируется временем прибытия наиболее медленного (или наиболее удаленного) компонента гетерогенной группы. Системная сложность управления заключается в том, что автолестницы составляют малую долю (порядка 20%) от общего парка техники, обладают сниженной маневренностью и увеличенными массогабаритными характеристиками, что делает их критически уязвимыми к суточным колебаниям пропускной способности улично-дорожной сети (УДС). Свойства и настройка популяции агентов изображены на рисунке 2.4.

fireTrucks - FireTruck

Имя: fireTrucks ☒ Отображать имя ☐ Исключить

☐ Одиночный агент ☒ Популяция агентов

Популяция: ☐ Изначально пуста ☒ Содержит заданное кол-во агентов ☐ Загружается из базы данных

Начальное количество агентов: 5

type: randomTrue(0.8) ? TANKER : LADDER

homeBase: randomFrom(allStations)

working: ☐

Рисунок 2.4 — настройка генерации гетерогенной популяции

Такой подход позволяет моделировать реалистичную хаотичную динамику одновременного возникновения множественных пожаров в разных районах города, создавая ситуации пиковой нагрузки и дефицита ресурсов для обучения нейросетевой модели [2].

Визуальная идентификация агентов на 2D-карте осуществляется за счет программного переопределения цвета заливки базового элемента анимации в зависимости от типа, согласно коду `this.type == TANKER ? red : blue`. Характеристики агентов представлены в таблице 2.1.

Таблица 2.1 — Параметры и тактическое назначение гетерогенных агентов
пожарной техники

Тип пожарного автомобиля	Идентификатор	Доля в парке	Визуализация на ГИС-карте	Тактическая роль в среде
Автоцистерна	TANKER	80%	Красный цвет (Red)	Базовый ресурс. Направляется на инциденты низкого ранга сложности (<i>Rank</i> = 1).
Автолестница	LADDER	20%	Синий цвет (Blue)	Дефицитный ресурс. Критически необходим для высотных инцидентов (<i>Rank</i> = 2).

2.2 Выбор типа и структуры нейросети, описание процесса сбора и обработки данных

2.2.1 Агрегация и векторизация состояния

Для обеспечения возможности обработки данных нейронной сетью, многомерная пространственная и ресурсная информация, поступающая от симулятора AnyLogic, преобразуется в единый плоский вектор (Flattened Vector) S_t . В программной реализации на Python за процесс векторизации отвечает специализированная функция обработки.

Вектор состояния включает в себя:

Ранг (сложность) текущего пожара (severity).

Вектор эвристических дистанций от очага возгорания до каждой из доступных пожарных станций (distances).

Матрицу состояний пожарных частей — наличие свободных единиц гетерогенной техники (автоцистерн и автолестниц) в каждой части (station_status).

За агрегацию данных и сборку вектора состояния с преобразованием параметров к типу данных `np.float32` для обеспечения совместимости с тензорными вычислениями в PyTorch [15] отвечает функция `flatten_state()` (Листинг 2.3)

```
def flatten_state(severity, distances, station_status):
    state = [severity]

    state.extend(distances)
    for status in station_status:
        state.extend(status)

    return np.array(state, dtype=np.float32)
```

Листинг 2.3 — Функция агрегации вектора состояния

2.2.2 Архитектура глубокой Q-сети (DQN)

В качестве математического аппарата для аппроксимации Q-функции (ожидаемой полезности действий) была выбрана архитектура глубокой полносвязной нейронной сети прямого распространения (Multi-Layer Perceptron). Разработка выполнена с использованием фреймворка PyTorch [15].

Архитектура модели DQN включает три линейных (полносвязных) слоя:

Входной слой (fc1): размерность динамически определяется длиной вектора состояния `input_dim`. Слой проецирует данные в скрытое пространство размерностью 64 нейрона.

Скрытый слой (fc2): содержит 64 нейрона.

Выходной слой (fc3): размерность соответствует количеству возможных действий `output_dim` (числу доступных пожарных станций). Каждый выходной нейрон выдает прогнозируемое Q-значение для соответствующей станции.

В качестве активационной функции скрытых слоев используется функция усеченного линейного блока ReLU (Rectified Linear Unit), математически определяемая как $f(x) = \max(0, x)$. ReLU предотвращает проблему затухания градиента при обратном распространении ошибки. Программная реализация класса полносвязной искусственной нейронной сети прямого распространения с

описанием структуры слоев fc1–fc3 и активационных функций ReLU на языке Python описана в листинге 2.4.

```
class DQN(nn.Module):  
    def __init__(self, input_dim, output_dim):  
        super(DQN, self).__init__()  
        self.fc1 = nn.Linear(input_dim, 64)  
        self.fc2 = nn.Linear(64, 64)  
        self.fc3 = nn.Linear(64, output_dim)  
  
    def forward(self, x):  
        x = torch.relu(self.fc1(x))  
        x = torch.relu(self.fc2(x))  
        return self.fc3(x)
```

Листинг 2.4 — Программная реализация класса сети

2.2.3 Политика выбора действий и расчет функции вознаграждения

Для балансировки между исследованием новых стратегий (Exploration) и использованием уже накопленного опыта (Exploitation) агент применяет ε -жадную политику (ε -greedy) [8].

При старте обучения параметр исследования $\varepsilon = 1.0$, что означает 100% случайный выбор пожарной станции. С каждым завершённым эпизодом (потушенным пожаром) значение ε экспоненциально уменьшается с коэффициентом затухания $\text{EPSILON_DECAY} = 0.995$ до минимального порога $\text{EPSILON_MIN} = 0.05$. Если случайное число превышает ε , сеть выполняет forward проход и выбирает станцию с максимальным предсказанным Q-значением.

$$A = \arg \max_A Q(S, A), \quad (2.1)$$

где A — выбранное нейросетевым агентом оптимальное действие (номер целевой пожарной части);

argmax — оператор аргумента максимума, возвращающий действие, которое максимизирует ценность стратегии;

$Q(S, A)$ — прогнозируемая функция ожидаемой полезности (Q-значение) для заданного состояния и действия;

S — текущее многомерное состояние имитационной среды (вектор состояния).

Функция вознаграждения агента (Reward) рассчитывается на основе реального времени в пути пожарной техники, полученного от имитационной модели после завершения вызова:

$$R = -T_{\text{пути}} \cdot n, \quad (2.2)$$

где R — скалярный показатель отклика (значение вознаграждения или штрафа) виртуальной среды;

$T_{\text{пути}}$ — фактическое время пожарной техники в пути, полученное от модели симуляции;

n — коэффициент скорости обучения ($n = 10$).

Математическая специфика данной постановки заключается в том, что индивидуальный жизненный цикл обслуживания каждого отдельного пожара представляет собой одношаговый процесс. Однако на уровне макрокоординации всего городского гарнизона система сохраняет фундаментальные свойства Марковского процесса принятия решений (MDP). Действие A_t , совершенное агентом в отношении инцидента в момент времени t (высылка техники из конкретной пожарной части), мгновенно и детерминированно изменяет глобальный вектор доступности гетерогенных ресурсов всего города. Данное изменение напрямую предопределяет распределение вероятностей для формирования вектора состояний среды S_{t+1} в следующий стохастический момент времени $t + 1$, когда произойдет генерация нового независимого инцидента. Таким образом, выполняется марковское свойство отсутствия последствия, где текущие ресурсы системы полностью определяют её будущее поведение.

Использование отрицательного множителя формирует штраф: чем дольше машина едет к пожару (например, из-за пробок или из-за вызова с дальней станции), тем больший штраф получает сеть. Это заставляет ИИ искать стратегии, минимизирующие интегральное время реагирования.

2.2.4 Механизм буфера памяти (Replay Buffer) и алгоритм обучения

Фундаментальной проблемой обучения нейросетей в динамических средах является сильная корреляция последовательных состояний. Для устранения данного эффекта используется механизм буфера воспроизведения опыта (Experience Replay) [11].

В программной реализации буфер memory организован на базе двусторонней очереди collections.deque с фиксированной максимальной длиной в 5000 переходов. Для корректного связывания решения о выезде с фактическим временем прибытия используется ассоциативный массив active_transitions, в котором временные состояния сохраняются по уникальному ключу fire_id.

Полный исходный код функции, реализующей сопоставление ID инцидента, расчет штрафных санкций и последующее пополнение циклического буфера памяти представлен в листинге 2.5.

```
def update_reward(fire_id, travel_time):
    global active_transitions, EPSILON, current_loss
    reward = -travel_time * 10.0

    # with open("dqn_high.csv", "a", encoding="utf-8") as
log_file:
    #     log_file.write(f"{fire_id},{travel_time}\n")

    if fire_id in active_transitions:
        transition = active_transitions.pop(fire_id)
        memory.append((transition['state'],
transition['action'], reward))
        print(f"    [PYTHON] Memory updated for fire {fire_id}.
Buffer: {len(memory)}/{memory.maxlen}")
        if EPSILON > EPSILON_MIN:
```



```

        EPSILON *= EPSILON_DECAY
    train_model()

```

Листинг 2.5 — Логика обновления вознаграждения и пополнения памяти

В рамках разработанного алгоритма обучения метод `train_model()` (Листинг 2.6) реализует оптимизацию параметров нейросети с учетом асинхронной природы возникновения городских инцидентов.

```

def train_model():
    global current_loss, current_reward

    if len(memory) < BATCH_SIZE:
        return

    batch = random.sample(memory, BATCH_SIZE)
    states, actions, rewards = zip(*batch)

    states = torch.FloatTensor(np.array(states))
    actions = torch.LongTensor(actions).unsqueeze(1)
    rewards = torch.FloatTensor(rewards).unsqueeze(1)

    q_values = model(states).gather(1, actions)

    loss = nn.MSELoss()(q_values, rewards)

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
    current_loss = loss.item()
    current_reward = rewards.mean().item()

```

Листинг 2.6 — Реализация функции `train_model`

Поскольку индивидуальные вызовы перекрываются во времени, сквозное многошаговое дисконтирование внутри одной заявки невозможно. Каждый переход, сохраняемый в циклическом буфере памяти `memory`, фиксирует терминальный шаг: после выполнения действия среда переходит в поглощающее состояние S_{t+1} , ценность которого по определению равна нулю: $V(S_{t+1}) = 0$.

Согласно теории динамического программирования Р. Беллмана, каноническое уравнение для целевого значения Q -функции (Target Q-value) имеет вид:

$$Y_t = R_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a'), \quad (2.3)$$

где Y_t — расчетное целевое значение функции ценности на текущем дискретном такте времени t ;

R_{t+1} — скалярная величина немедленного вознаграждения (штрафа), формируемая имитационной средой на такте $t + 1$ в ответ на совершенное агентом действие;

γ — коэффициент дисконтирования ($\gamma \in [0,1]$), определяющий степень приоритета долгосрочных будущих наград над немедленными;

$\max_{a'}$ — оператор нахождения максимума по множеству допустимых альтернативных действий;

s_{t+1} — вектор следующего состояния имитационной среды, в которое система переходит в результате выполнения текущего действия;

a' — переменная, обозначающая множество всех возможных вариантов будущих действий агента в состоянии s_{t+1} ;

$Q(s_{t+1}, a')$ — прогнозируемая функция ожидаемой полезности (Q -значение) для следующего состояния среды и каждого из возможных будущих действий.

В условиях, когда текущее состояние является терминальным, будущая ценность $\max_{a'} Q(s_{t+1}, a')$ тождественно обращается в ноль, в связи с чем целевое значение функции ценности сводится к значению немедленного скалярного штрафа среды:

$$Y_t = R_{t+1}. \quad (2.4)$$

Оценка расхождения между текущими предсказаниями модели и редуцированными целевыми значениями Беллмана автоматизирована в PyTorch с помощью критерия среднеквадратичной ошибки (MSELoss) по формуле:

$$L = \frac{1}{B} \sum_{i=1}^B (Q(s_i, a_i) - R_i)^2, \quad (2.5)$$

где L — функция потерь;

B — размер пакета данных (`BATCH_SIZE = 32`);

$Q(s_i, a_i)$ — оценка полезности выбранного действия a_i для состояния s_i , возвращаемая выходным слоем сети;

R_i — фактический штраф за время нахождения техники в пути.

Вычисление градиентов методом обратного распространения ошибки `loss.backward()` и последующая корректировка весовых коэффициентов оптимизатором `optim.Adam()` с шагом $LR = 0.001$ минимизируют данную функцию потерь, настраивая аппроксиматор на прямую генерацию точных оценок временных затрат.

Для интеграции с панелью администратора в модуле предусмотрены методы сериализации обученной модели `save_weights()` (сохраняющие тензоры весов в бинарный файл `dqn_weights.pth`) и `load_weights()`, позволяющие мгновенно развернуть готовую СППР без необходимости повторного обучения.

Графическое подтверждение сходимости обучения, выражающееся в стабильном снижении функции потерь (Loss) и сопутствующем уменьшении исследовательского фактора (Epsilon) на графиках панели администратора, показано на рисунке 2.5.

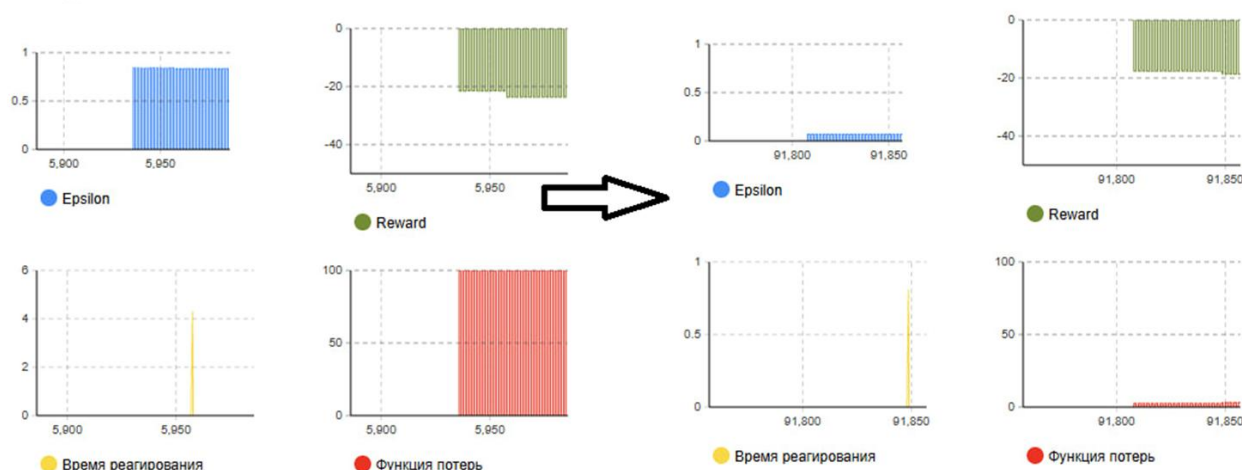


Рисунок 2.5 — Скриншот панели администратора, демонстрирующий процесс сходимости функции потерь (Loss) и падения параметра Epsilon в реальном времени

2.3 Разработка пользовательских интерфейсов

Внедрение интеллектуальных алгоритмов глубокого обучения в контур управления экстренными службами требует разработки эргономичного человеко-машинного интерфейса (HMI). Пользователи системы обладают различным уровнем технической подготовки и решают принципиально разные оперативные задачи. С целью обеспечения максимальной эффективности взаимодействия в рамках программного комплекса спроектировано два изолированных графических интерфейса: рабочее место лица, принимающего решения (оперативного диспетчера), и панель мониторинга для службы технической поддержки (администратора модели).

2.3.1 Интерфейс для лица, принимающего решение (оперативного диспетчера)

Главной задачей оперативного диспетчера является быстрая верификация поступающих вызовов и безошибочная координация боевых расчетов. Учитывая высокий уровень психологического стресса и предельно жесткие временные лимиты, в рамках которых принимаются стратегические решения в экстренных службах, интерфейс спроектирован с учетом строгих требований когнитивной эргономики. Основным принцип построения данного интерфейса — тотальная

минимизация информационного шума и предотвращение перегрузки внимания оператора. На экран выводятся исключительно те оперативные метрики и функциональные элементы, которые критически необходимы для контроля над дорожной ситуацией и валидации предложений ИИ-помощника.

Центральным элементом интерфейса диспетчера выступает масштабируемая **ГИС-карта**, на которой в режиме реального времени и с высокой степенью топографической точности отображаются все ключевые события виртуальной среды:

Визуализация инцидентов: при генерации нового вызова системой на ГИС-карте, строго в координатах очага возгорания, мгновенно отрисовывается специализированный графический маркер (fireIcon). Строгая пространственная привязка иконки инцидента позволяет оператору за доли секунды визуально оценить локализацию чрезвычайной ситуации, плотность окружающей городской застройки и эвристическую удаленность пожара от зон территориальной ответственности дежурных пожарных частей.

Отображение маршрутов и отслеживание техники: процесс следования назначенных боевых расчетов к месту ЧС динамически анимирован поверх реального графа участков улично-дорожной сети. Для обеспечения мгновенной читаемости тактического потенциала выехавших машин, в системе реализована цветовая дифференциация гетерогенного автопарка. Базовые автоцистерны, выполняющие универсальные задачи тушения, отображаются красным цветом, в то время как дефицитные автолестницы, требуемые для сложных высотных пожаров — синим (с использованием тернарного оператора `this.type == TANKER ? red : blue`). Такое визуальное кодирование гарантирует, что диспетчер с первого взгляда, не обращаясь к дополнительным табличным сводкам, понимает состав и специализацию направленной группировки.

Помимо инструментов визуального мониторинга ГИС, интерфейс оснащен интерактивной **информационно-рекомендательной панелью СППР**, закрепленной в зоне быстрого периферийного доступа. Данный модуль выступает ключевым связующим звеном между человеческим интеллектом

оператора и алгоритмом машинного обучения. На панели непрерывно агрегируются и обновляются данные телеметрии, собираемые головным агентом (Main):

Индикация текущего статуса загруженности каждой из четырех пожарных станций, включая точный численный мониторинг доступного резерва техники (раздельно для АЦ и АЛ). Это помогает диспетчеру заблаговременно распознавать надвигающийся структурный дефицит в определенных районах мегаполиса.

Прямая рекомендация нейросетевого модуля (AI Overlay), выделяющая оптимальное депо для обслуживания текущего вызова, основанная на предиктивном расчете минимальных штрафов и оценке пробок.

Интерактивный переключатель режима управления (Manual / Auto). Наличие данного элемента управления принципиально важно для внедрения интеллектуальных систем в реальную практику (концепция Human-in-the-Loop): ИИ выступает исключительно в роли интеллектуального советника, но оператор сохраняет за собой возможность директивного (ручного) перехвата управления. Если диспетчер, основываясь на собственном опыте или внешних данных, оценивает риск ИИ-решения как неприемлемый, он имеет полное право проигнорировать подсказку нейросети и назначить технику самостоятельно, сохраняя за собой окончательную юридическую и оперативную ответственность за исход ликвидации ЧС.

Общий вид разработанного графического интерфейса рабочего места оперативного диспетчера, содержащий интерактивную ГИС-карту с динамической визуализацией маршрутов следования (Рисунок 2.6) и рекомендательную панель СППР (Рисунок 2.7).

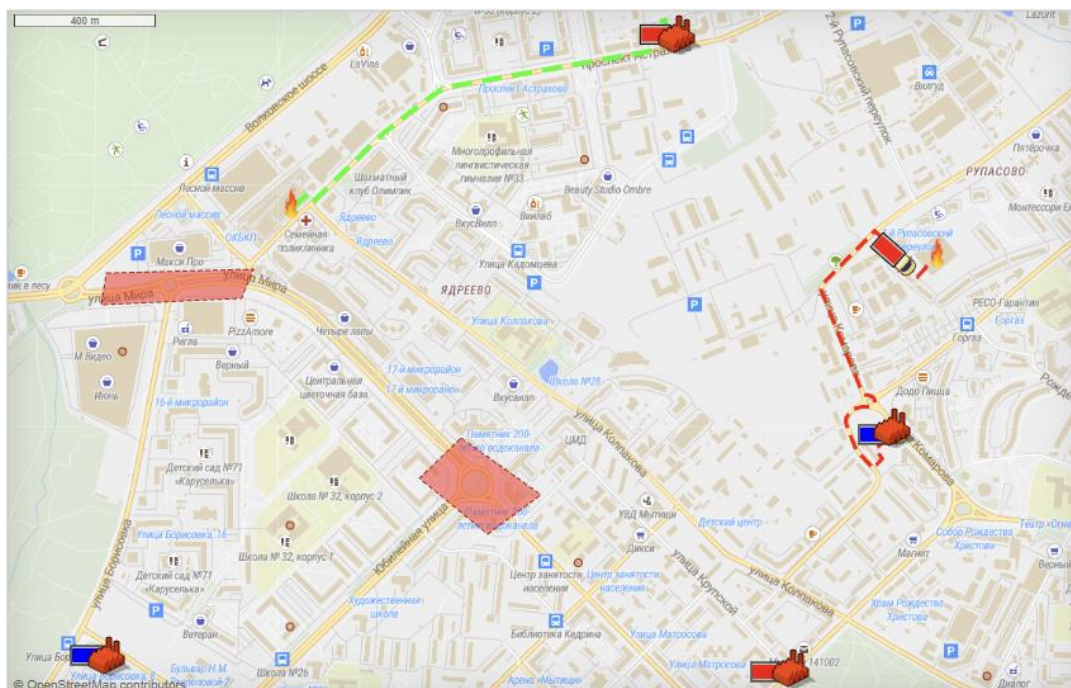


Рисунок 2.6 — Интерактивная ГИС-карта с динамической визуализацией маршрутов следования



Рисунок 2.7 — Рекомендательная панель СППР для диспетчера

2.3.2 Интерфейс службы поддержки и администрирования модели

Интерфейс службы технической поддержки (панель администратора) скрыт от рядового диспетчера и предназначен для инженеров по данным и системных администраторов, обслуживающих работу СППР. Основная цель данного интерфейса — обеспечение процессов тонкой настройки (Fine-tuning),

дообучения модели без перезапуска симулятора и мониторинга качества работы ИИ.

Панель администратора структурно разделена на два функциональных блока:

Блок управления стресс-сценариями. Содержит ползунки (Slider) регулировки вероятности возникновения пожара p , коэффициента загруженности дорог k , скорости передвижения автотехники v . Изменяя данные параметры, администратор может в реальном времени создавать условия искусственного кризиса, проверяя реакцию нейросети на пиковый дефицит автолестниц.

Блок управления жизненным циклом нейросети. Включает в себя элементы для прямой отправки управляющих команд через интеграционный шлюз Rypeline в скрипт Python: Кнопка «Сохранить веса» (`save_weights()`) — инициирует сериализацию текущих обученных параметров нейросети в файл `dqn_weights.pth` для создания резервной копии удачного эксперимента. Кнопка «Загрузить модель» (`load_weights()`) — выполняет чтение сохраненных весов и принудительно сбрасывает коэффициент исследования ϵ к значению `EPSILON_MIN`. Это позволяет мгновенно перевести систему из режима длительного случайного обучения в режим жесткой эксплуатации обученного опыта (Exploitation).

Для обеспечения прозрачности процесса обучения в интерфейс интегрированы динамические графики мониторинга качества работы (телеметрия потерь и наград). Функция `get_metrics()` с определенной частотой опрашивает вычислительное ядро Python и возвращает в AnyLogic строку формата `current_loss, EPSILON, current_reward`. Симулятор парсит эту строку и отрисовывает графики сходимости ошибки обучения (Loss Function) и роста среднего вознаграждения агента (Average Reward), что позволяет инженеру объективно оценивать готовность модели к внедрению в боевой режим.

Визуальное представление разработанной панели интерфейса администратора модели, объединяющей органы управления интенсивностью потока инцидентов, кнопки сохранения/загрузки весов и графики оперативной телеметрии обучения, изображено на рисунке 2.8.

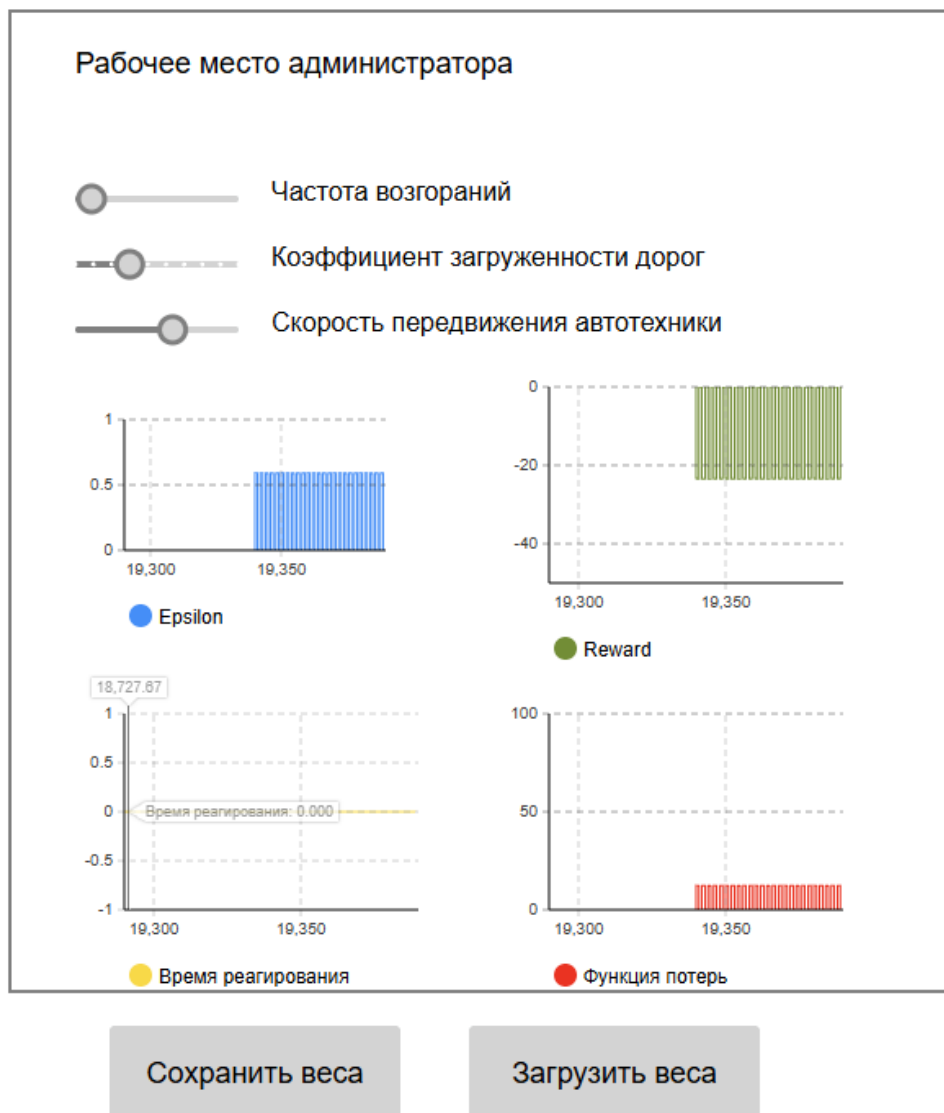


Рисунок 2.8 — Панель интерфейса администратора

2.4 Реализация и обучение нейросетевой модели на тестовых данных

Процесс практической реализации и обучения нейросетевого агента осуществлялся в синтетической имитационной среде при непрерывной стохастической генерации тестовых инцидентов. Вычислительный процесс инициализировался с абсолютно случайных весовых коэффициентов нейронной сети, что имитировало отсутствие априорного знания о топологии городского трафика и дефиците техники.

Обучение модели можно условно разделить на две ключевые фазы:

Фаза исследования среды (Exploration). На начальных этапах симуляции параметр жадности ϵ удерживался в диапазоне от 1.0 до 0.5. В этот период система целенаправленно совершала случайные или субоптимальные действия (например, отправляла автолестницы на инциденты первого ранга). Данная тактика искусственно провоцировала ресурсные кризисы, позволяя наполнить буфер памяти (memory deque) критическим негативным опытом и высокими штрафами за задержки в пути. Алгоритм не выполнял оптимизацию весов до тех пор, пока буфер памяти не аккумулировал минимальный пакет переходов, равный `BATCH_SIZE = 32`.

Фаза эксплуатации и стабилизации (Exploitation & Convergence). По мере накопления опыта и экспоненциального снижения параметра ϵ (с шагом 0.995 за каждый обработанный вызов), нейронная сеть начала доверять собственным аппроксимациям функции ценности $Q(S, A)$. Обучение градиентным спуском (оптимизатор Adam) приводило к постепенному снижению среднеквадратичной ошибки (Loss).

В процессе обучения на тестовых данных было зафиксировано, что после прохождения агентом порядка 1500–2000 симулированных вызовов метрика среднего вознаграждения (Average Reward) выходит на стабильное плато. Это свидетельствует о том, что нейросеть успешно выявила скрытые паттерны среды: алгоритм перестал назначать дефицитную технику (LADDER) на простые возгорания, резервируя её для потенциальных высотных инцидентов ($Rank = 2$), и научился распределять нагрузку между всеми четырьмя депо более равномерно.

Графическое подтверждение сходимости обучения, выражающееся в стабильном снижении функции потерь (Loss) и сопутствующем поэтапном выходе на стабильное плато функции вознаграждения (Average Reward) в процессе имитационного прогона, показано на рисунке 2.9.



Рисунок 2.9 — Графики обучения нейросети: падение Loss-функции и стабилизация функции награды

2.5 Критерии оценки качества маршрутизации и интерфейсных решений

Оценка эффективности разработанного программного комплекса не может базироваться исключительно на метриках машинного обучения (Loss, Reward), так как они не отражают оперативного смысла решаемой задачи. Для объективного сравнительного анализа разработанной нейросетевой модели с традиционными алгоритмами (в частности, жадным алгоритмом поиска кратчайшего пути), были сформулированы следующие прикладные критерии оценки качества:

Интегральное время реагирования (Критерий скорости): Ключевая метрика, определяемая как среднее время, затрачиваемое целевой оперативной единицей на прибытие к месту ЧС от момента поступления вызова. Оценка качества считается высокой, если дисперсия времени реагирования снижается, а математическое ожидание не превышает установленных регламентных нормативов (10 минут для городской среды).

Коэффициент сохранности дефицитного резерва (Критерий надежности): Введенный в рамках данного исследования параметр, оценивающий способность системы избегать “вымывания” специализированной техники. Он рассчитывается как процент успешных назначений автолестниц (LADDER) строго на высотные пожары ($Rank = 2$) без ситуаций, когда необходимая техника отсутствует на базах из-за ошибочного распределения на простые вызовы.

Эргономика и отклик интерфейсных решений (UI/UX критерии): Оценка интерфейсного уровня (для диспетчера и администратора) проводится на основе критерия безотказности работы (отсутствие зависаний графического потока AnyLogic при расчетах тензоров в Python). Архитектура Pipeline позволяет свести задержку “замораживания” симуляции (Simulation Time Freezing) к величинам менее 50 миллисекунд, что делает процесс работы ИИ абсолютно незаметным для диспетчера и обеспечивает бесшовный пользовательский опыт (Seamless UX).

2.6 Методология, условия проведения экспериментов и программный комплекс

Целью экспериментальных исследований является оценка эффективности разработанной нейросетевой модели поддержки принятия решений (СППР) на базе алгоритма глубокого Q -обучения (DQN) при маршрутизации пожарной автотехники в ГИС-среде города по сравнению с традиционными диспетчерскими эвристиками.

Для проведения вычислительных экспериментов была использована ранее разработанная имитационная система, функционирующая в среде AnyLogic.

Для обеспечения чистоты эксперимента были приняты следующие методологические условия:

1. Контроль и фиксация случайности в среде моделирования AnyLogic: В имитационной модели AnyLogic было настроено фиксированное начальное число генератора случайных чисел ($seed = 1$). Это гарантирует, что

последовательность независимых испытаний Бернулли, определяющая генерацию инцидентов (географические координаты очагов, временные интервалы между пожарами и требования к типам привлекаемой техники), будет абсолютно идентичной во всех сравниваемых сессиях моделирования.

2. Контроль и фиксация случайности в программной среде Python:

Для обеспечения повторяемости процессов инициализации весов нейросети, формирования буфера памяти и стохастического выбора действий агентом в коде на языке Python была реализована комплексная функция фиксации генераторов псевдослучайных чисел `seed_everything` (Листинг 2.7).

```
def seed_everything(seed):  
    random.seed(seed)  
    os.environ['PYTHONHASHSEED'] = str(seed)  
    np.random.seed(seed)  
    torch.manual_seed(seed)  
    if torch.cuda.is_available():  
        torch.cuda.manual_seed(seed)  
        torch.cuda.manual_seed_all(seed)  
    torch.backends.cudnn.deterministic = True  
    torch.backends.cudnn.benchmark = False
```

**Листинг 2.7 — Реализация функции фиксации генераторов
псевдослучайных чисел**

При проведении испытаний данная функция (Листинг 2.7) вызывалась со значением `seed = 1`.

Научно-методическое обоснование: стоит отметить, что равенство числовых значений инициализирующих последовательностей в AnyLogic и Python не является обязательным условием, поскольку они управляют независимыми программными контекстами (Java Virtual Machine и PyTorch/NumPy соответственно) и не влияют на физический смысл сравнительного анализа. Главным и достаточным условием чистоты эксперимента является независимая фиксация начальных значений (`seeds`) для

каждого из инструментов, что гарантирует полную детерминированность и воспроизводимость результатов при повторных запусках.

3. Схема проведения испытаний: для сбора экспериментального материала программа запускалась последовательно 4 раза по следующей схеме:

- Запуск № 1: Базовый алгоритм диспетчеризации функционирует в условиях сценария «Штатный режим».
- Запуск № 2: Базовый алгоритм диспетчеризации функционирует в условиях сценария «Высокая нагрузка».
- Запуск № 3: Разработанная СППР под управлением обученного DQN-агента функционирует в условиях сценария «Штатный режим».
- Запуск № 4: Разработанная СППР под управлением обученного DQN-агента функционирует в условиях сценария «Высокая нагрузка».

Использование фиксированного `seed` в Python гарантирует, что при повторном запуске нейросетевого агента (Запуски №3 и №4) модель будет выдавать строго одинаковые управляющие решения, исключая флуктуации результатов.

4. Объем экспериментальной выборки: в каждом из 4 запусков объем выборки составил ровно 6000 последовательных измерений. Данный объем выборки является репрезентативным и достаточным для проведения статистического анализа.

5. Идентичность информационного базиса: оба алгоритма функционируют в абсолютно одинаковых условиях. Нейросетевой агент и базовый эвристический алгоритм получают на вход одинаковую матрицу евклидовых расстояний (прямых линий от пожарных депо до точки инцидента), что ставит их в равные условия с точки зрения исходной информированности о пространстве.

6. Идентичность гетерогенной структуры автопарка: в рамках каждого из 4 запусков состав, численность и распределение доступных пожарных автомобилей по депо были абсолютно одинаковыми. Общая структура автопарка

в имитационной модели строго зафиксирована в пропорции: 80% основных пожарных автомобилей (автоцистерн — АЦ / TANKER) и 20% специальных дефицитных пожарных автомобилей (автолестниц — АЛ / LADDER). Это условие гарантирует, что оба алгоритма функционировали под воздействием одинаковых ресурсных ограничений и одинаковой степени дефицита тяжелой спецтехники.

Для проведения тестирования и симуляции различных режимов нагрузки на городскую инфраструктуру в работе использовался ранее разработанный интерфейс администратора системы. С его помощью производилось изменение параметров внешней среды моделирования с использованием трех интерактивных элементов управления (слайдеров):

1. Вероятность возникновения пожара (p): регулирует интенсивность входящего стохастического потока событий в интервале $[0.0; 1.0]$.

2. Коэффициент загруженности дорожной сети (k): определяет уровень плотности транспортных потоков и заторов на графе дорог в условной шкале от 1 до 10.

3. Базовая скорость передвижения техники (v): устанавливает номинальный скоростной режим пожарных автомобилей с учетом погодных и дорожных факторов в интервале от 1 до 100 условных единиц.

2.7 Сценарии моделирования

Сравнительный анализ алгоритмов проводился в рамках двух контрастных экспериментальных сценариев, отражающих полярные режимы функционирования экстренных служб, параметры которых задавались посредством интерфейса администратора:

Сценарий № 1. Функционирование в штатном режиме

Данный сценарий характеризуется умеренной нагрузкой на оперативную инфраструктуру, относительно свободной дорожной сетью и благоприятными климатическими условиями. На административной панели управления были выставлены следующие параметры:

- Вероятность возникновения пожара: $p = 0,2$;

- Коэффициент загруженности дорог: $k = 4$;
- Скорость передвижения пожарной техники: $v = 60$.

Сценарий № 2. Функционирование в жестких (экстремальных) условиях

Сценарий имитирует критическое состояние городской среды, характерное для часов пик в сочетании с неблагоприятными погодными условиями и резким ростом числа чрезвычайных ситуаций. На административной панели установлены следующие параметры:

- Вероятность возникновения пожара: $p = 0,6$;
- Коэффициент загруженности дорог: $k = 8$;
- Скорость передвижения пожарной техники: $v = 40$.

2.8 Описание сравниваемых алгоритмов и принцип обучения нейросети

В рамках проводимых испытаний исследовалось поведение двух противоположных стратегий диспетчеризации:

1. Базовый алгоритм (ближайший по радиусу): представляет собой классический пространственный поиск с проверкой наличия свободного ресурса в депо (Листинг 2.8).

```
double minDistance = Double.MAX_VALUE;
for (int i = 0; i < allStations.size(); i++) {
    double dist = allStations.get(i).distance(pt);
    boolean hasFreeTruck = false;
    for (FireTruck truck: fireTrucks) {
        if (!truck.working && !reservedThisMinute.contains(truck)
        && truck.homeBase == allStations.get(i)) {
            hasFreeTruck = true;
            break;
        }
    }
    if (hasFreeTruck && dist < minDistance) {
        minDistance = dist;
        chosen_station_index = i;
    }
}
```



```

    }
}
if (chosen_station_index != -1) {
    for (FireTruck truck: fireTrucks) {
        if (!truck.working && !reservedThisMinute.contains(truck)
&& truck.homeBase == allStations.get(chosen_station_index)) {
            selectedTruck = truck;
            break;
        }
    }
}
}

```

Листинг 2.8 — Алгоритм поиска ближайшего депо с учетом доступности техники

При возникновении инцидента алгоритм осуществляет расчет расстояния по прямой линии от географической точки пожара до каждого из доступных пожарных депо. В среде имитационного моделирования AnyLogic для этого вычисления была задействована встроенная функция определения расстояния на ГИС-карте между двумя пространственными точками (координатами инцидента и точкой дислокации пожарного депо) [16]. После чего алгоритм отправляет команду на выезд экипажа из депо с минимальным расстоянием, в котором на данный момент имеется хотя бы одна доступная пожарная автотехника (любого типа).

Особенность базового алгоритма: как видно из листинга 2.8, базовая стратегия является «слепой» по отношению к типам пожарных автомобилей. Она проверяет лишь общее наличие свободной техники на базе (`hasFreeTruck = true`) и не дифференцирует депо по количеству оставшихся автоцистерн или автолестниц. Таким образом, базовый алгоритм без колебаний отправит дефицитную автолестницу на вызов низкой категории сложности, если данное депо оказалось географически ближе остальных.

2. Нейросетевой алгоритм (разработанный DQN-агент): принимает решения на основе вектора состояния S_t , содержащего аналогичную матрицу

евклидовых расстояний до депо, а также статус занятости гетерогенного автопарка.

Перед вызовом управляющего скрипта имитационная модель AnyLogic динамически собирает текущие параметры системы: вычисляет расстояния по прямой до всех депо, осуществляет отдельный подсчет доступных автоцистерн (TANKER) и автолестниц (LADDER) на каждой из баз, а затем сериализует эти структуры данных в строки для передачи через шлюз Pipeline (Листинг 2.9).

```
int numStations = allStations.size();
double[] distances = new double[numStations];
int[][] stationStatus = new int[numStations][2];

for (int i = 0; i < numStations; i++) {
    distances[i] = allStations.get(i).distance(pt);
    int countTanker = 0;
    int countLadder = 0;
    for (FireTruck truck : fireTrucks) {
        if (!truck.working
&& !reservedThisMinute.contains(truck) && truck.homeBase
== allStations.get(i)) {
            if (truck.type == TANKER) countTanker++;
            else if (truck.type == LADDER)
countLadder++;
        }
    }
    stationStatus[i][0] = countTanker;
    stationStatus[i][1] = countLadder;
}

String strDist = java.util.Arrays.toString(distances);
String strStatus =
java.util.Arrays.deepToString(stationStatus);

String pyCall = "get_action(" + currentFireId + ", " +
severity + ", " + strDist + ", " + strStatus + ")";
chosen_station_index = py.runResults(int.class, pyCall);
```

```
ai_recommendation = chosen_station_index;
```

Листинг 2.9 — Формирование вектора состояния и вызов нейросетевого DQN-агента

Особенность нейросетевого алгоритма: в отличие от эвристики, DQN-агент получает на вход двумерный массив `stationStatus`, хранящий точные остатки цистерн и лестниц по каждой станции отдельно. Имея перед собой полную картину доступности гетерогенных ресурсов, нейросеть обучается оценивать риски исчерпания редкого типа техники в конкретных районах города. Первоначально нейросеть получает на вход ту же «абстрактную» матрицу прямых расстояний, что и базовый алгоритм, и совершает случайные действия. В процессе обучения градиентный спуск минимизирует отклонение предсказанных сетью значений от реального времени пути, зафиксированного симулятором AnyLogic. Так как целевое значение функции ценности свелось к величине немедленного штрафа R_i , выходные нейроны сети в процессе сходимости алгоритма DQN начинают аппроксимировать математическое ожидание штрафной функции для каждой конкретной пожарной части. Таким образом, нейросетевой алгоритм осуществляет мгновенный предиктивный расчет рисков задержки, превосходя «слепой» пространственный поиск базовой эвристики за счет накопленного опыта о скрытом дефиците машин и топологии пробок

Происходит качественное изменение восприятия среды агентом:

- **Интеграция топологии дорожной сети:** Агент обучается понимать, что наименьшее расстояние по прямой не всегда является кратчайшим маршрутом.
- **Учет динамики заторов:** Агент адаптируется к штрафным зонам (`jamZones`), выучивая скрытые закономерности распределения заторов.
- **Резервирование дефицитных ресурсов:** Обучившись, DQN-агент избегает отправки редкой специализированной техники (LADDER) на простые пожары первого ранга, даже если депо с АЦ географически ближе всего к инциденту, сохраняя её для потенциально сложных высотных вызовов.

Таким образом, за счет балансирования действий нейросети функцией награды удастся привить агенту неявное «знание местности» и понимание системных взаимосвязей гетерогенной городской среды, не прибегая к ресурсоемким процедурам динамического поиска путей по графу дорог для всех пожарных частей в момент принятия решения диспетчером.

2.9 Анализ результатов вычислительных экспериментов

Анализ эффективности алгоритмов основывается на статистической обработке накопленных CSV-логов по результатам 6000 измерений для каждого из 4 запусков. Численные результаты сравнительного анализа для всех сценариев и алгоритмов диспетчеризации сведены в таблицу 2.2.

Таблица 2.2 — Сводные статистические показатели времени реагирования, мин

Алгоритм	Мат. ожидание, $M[T]$	Медиана, Me	Дисперсия, $D[T]$	СКО, σ	Нарушение регламента, %
Жадный (Низкая нагрузка)	6.27	3.99	22.89	4.78	26.26%
СППР DQN (Низкая нагрузка)	3.08	3.99	22.89	4.78	1.51%
Жадный (Высокая нагрузка)	12.68	7.25	117.60	10.84	40.44%
СППР DQN (Высокая нагрузка)	4.82	4.41	11.95	3.46	3.70%

2.9.1 Анализ Сценария № 1 (Низкая нагрузка)

В рамках первого сценария разработанная СППР DQN демонстрирует выраженное превосходство над жадным алгоритмом по всем оцениваемым статистическим критериям.

Согласно полученным данным, математическое ожидание времени доставки техники при использовании СППР составляет 3.08 мин, что в 2.03 раза (или на 50.88%) меньше, чем у жадного алгоритма (6.27 мин). Медианное значение времени реагирования также снизилось с 3.99 мин до 2.62 мин.

Графики плотности распределения частоты и функции кумулятивного распределения времени реагирования при условиях низкой нагрузки представлены на рисунках 2.10 и 2.11.

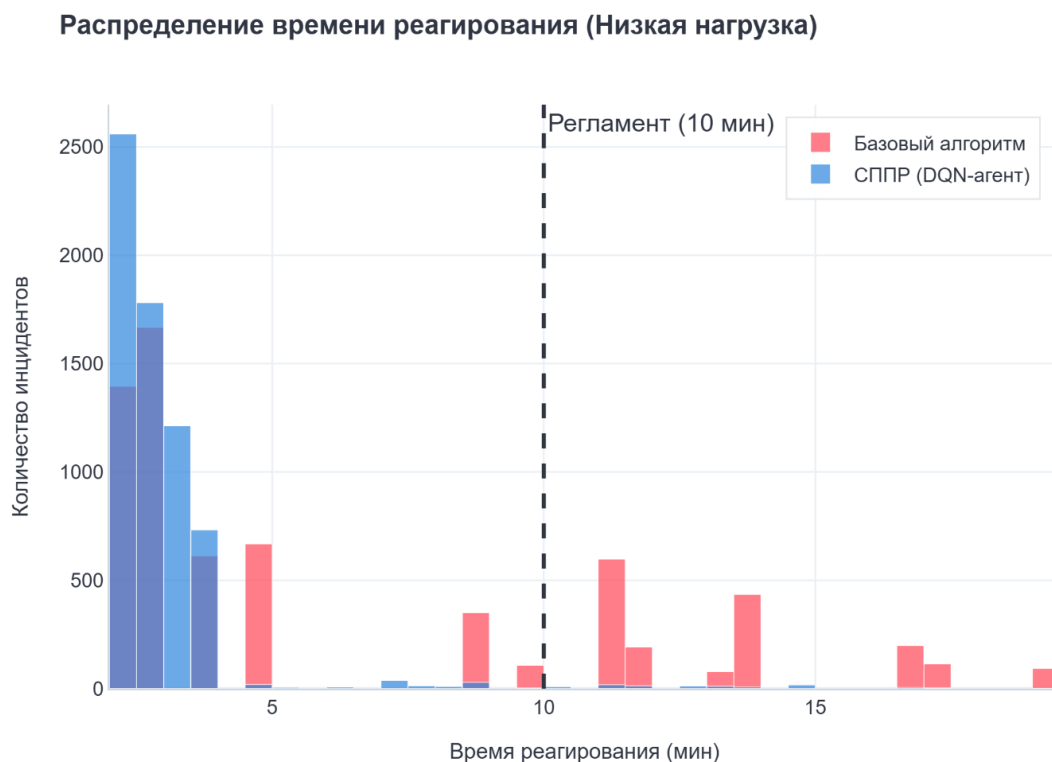


Рисунок 2.10 — Гистограмма плотности распределения времени реагирования при низкой нагрузке



Рисунок 2.11 — График кумулятивной функции распределения времени реагирования при низкой нагрузке

Анализ плотности распределения (рисунок 2.10) показывает, что кривая для СППР DQN имеет форму с ярко выраженной правосторонней асимметрией и острым пиком в диапазоне 2.0 - 3.5 минут. Напротив, распределение жадного алгоритма сильно размыто по оси абсцисс, обладая длинным правым плечом, уходящим в область больших временных задержек.

Важнейшим индикатором стабильности системы выступает дисперсия $D[T]$ (и соответствующее среднее квадратическое отклонение σ):

Для жадного алгоритма дисперсия составляет 22.89 мин^2 ($\sigma = 4.78 \text{ мин}$);

Для СППР DQN дисперсия снижается до 2.36 мин^2 ($\sigma = 1.54 \text{ мин}$).

Такое резкое сокращение дисперсии математически доказывает стабилизирующий эффект нейросети: решения по распределению вызовов становятся строго детерминированными и предсказуемыми.

Анализ кумулятивной функции распределения (рисунок 2.11) демонстрирует, что интегральная вероятность прибытия в нормативное время (до 10 минут согласно ФЗ-123 [9]) для разработанной СППР составляет 98.49%

(доля нарушений — всего 1.51%). Для базового жадного алгоритма вероятность успешного прибытия в регламентный срок составляет лишь 73.74%, а доля нарушений достигает критических для штатного режима 26.26%. 90-й перцентиль для СППР локализован на отметке 3.99 мин, тогда как у базовой эвристики он равен 13.57 мин, что превышает законный предел экстренного реагирования.

2.9.2 Анализ Сценария № 2 (Высокая нагрузка)

В условиях высокой нагрузки, когда стохастически возникает дефицит ресурсов, преимущества разработанной нейросетевой модели проявляются наиболее масштабно.

Математическое ожидание времени прибытия под управлением жадного алгоритма возрастает до критических 12.68 мин, что напрямую нарушает федеральные нормы безопасности. Интеллектуальная СППР DQN успешно удерживает данный показатель в безопасном диапазоне — 4.82 мин, демонстрируя сокращение времени доставки на 61.98%. Медиана для DQN составляет 4.41 мин, в то время как у базовой эвристики она равна 7.25 мин.

Графики плотности распределения частоты и функции кумулятивного распределения времени реагирования при условиях высокой нагрузки представлены на рисунках 2.12 и 2.13.

Распределение времени реагирования (Высокая нагрузка)

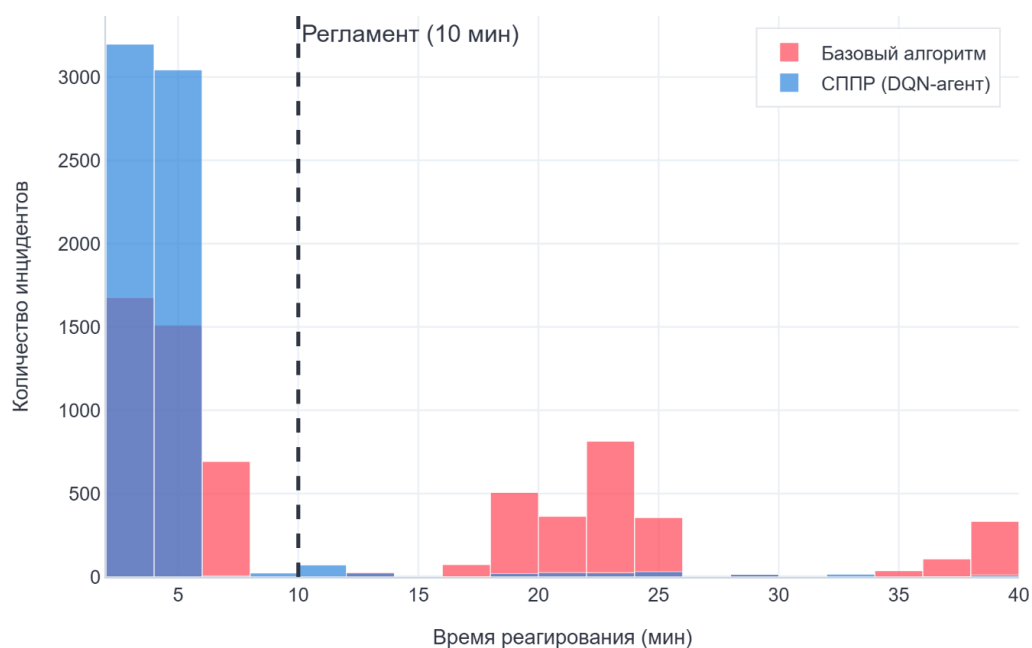


Рисунок 2.12 — Гистограмма плотности распределения времени реагирования при высокой нагрузке

Кумулятивное распределение времени реагирования (Высокая нагрузка)

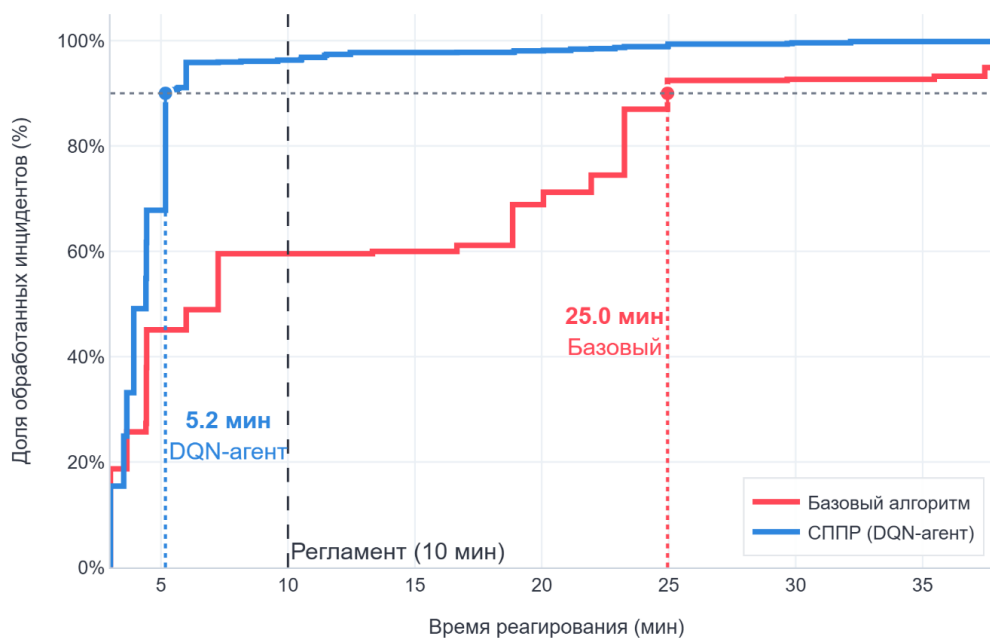


Рисунок 2.13 — График кумулятивной функции распределения времени реагирования при высокой нагрузке

Плотность распределения жадного алгоритма в стрессовых условиях (рисунок 2.12) полностью уплощается и приобретает тяжелый длинный «хвост» распределения, простирающийся вплоть до максимума в 38.14 мин. Дисперсия базового алгоритма увеличивается до катастрофических 117.60 мин² ($\sigma = 10.84$ мин). В реальных условиях диспетчеризации это означает полную потерю контроля над ситуацией со стороны оператора, когда время прибытия машины становится случайной величиной с огромным разбросом.

СППР DQN эффективно противодействует факторам неопределенности: дисперсия удерживается на уровне 11.95 мин² ($\sigma = 3.46$ мин). Кумулятивная функция (рисунок 2.13) подтверждает высокую надежность нейросетевого управления: доля нарушений регламента у DQN составляет 3.7%, тогда как у жадного алгоритма регламент нарушается в 40.44% случаев (почти каждый второй вызов обслуживается с опозданием).

90-й перцентиль времени прибытия для DQN составляет 5.18 мин, что свидетельствует о гарантированной доставке боевого расчета менее чем за 5.2 минуты в 90% всех чрезвычайных ситуаций. У жадного алгоритма этот показатель равен 24.95 мин, что делает ликвидацию пожаров на стадии их свободного развития практически невозможной.

2.9.3 Обобщенный сравнительный анализ эффективности и стабильности

Для комплексной интегральной оценки результатов моделирования по всем 4 запускам были построены обобщающие диаграммы. Сравнение размаха и стабильности времени реагирования с помощью диаграмм «ящик с усами» (диаграмм размаха) приведено на рисунке 2.14.

Сравнение размаха и стабильности времени реагирования

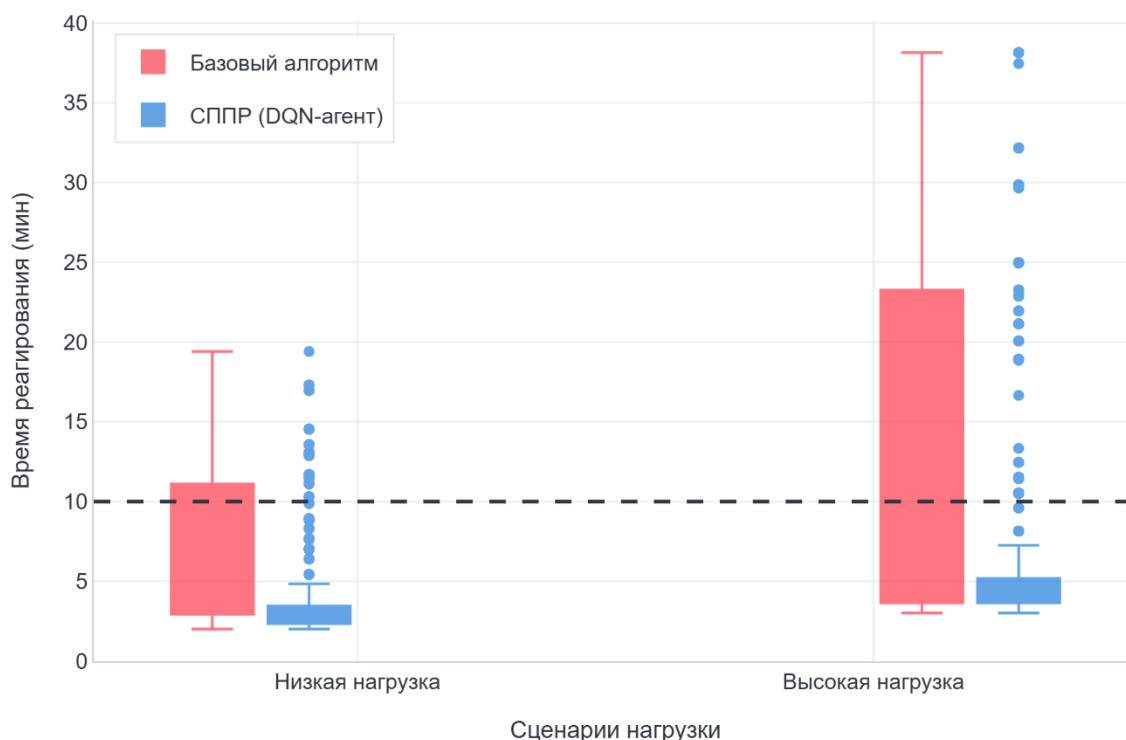


Рисунок 2.14 — Диаграмма размаха времени реагирования жадного и DQN алгоритма

Диаграмма размаха (рисунок 2.14) наглядно иллюстрирует динамику изменения квартилей:

Межквартильный размах (тело «ящика») для СППР DQN в обоих сценариях сжат в узких пределах (низкая нагрузка: [2.2, 3.4] мин; высокая нагрузка: [3.2, 5.0] мин). Это подтверждает высокую однородность принимаемых решений.

1. Для жадного алгоритма межквартильный размах существенно шире, а верхний ус диаграммы при высокой нагрузке уходит далеко за предел в 20 минут.

2. Единичные выбросы (точки за пределами усов) у DQN присутствуют, однако их плотность незначительна. В то же время у жадного алгоритма область за пределами усов перенасыщена аномально высокими значениями времени доставки, вызванными дефицитом техники и нерациональной логистикой.

Для сопоставления обобщенных средних характеристик была построена гистограмма среднего времени реагирования (рисунок 2.15).



Рисунок 2.15 — Гистограмма среднего времени реагирования

Гистограмма наглядно демонстрирует стабильно низкий уровень средних временных затрат при использовании интеллектуального DQN-агента. Даже в условиях жесткого стресс-сценария среднее время прибытия техники под управлением нейросети (4.82 мин) оказывается ниже, чем время прибытия базового алгоритма в штатном ненагруженном режиме (6.27 мин). Это доказывает высокую адаптивную способность модели.

Итоговое сопоставление важнейшего социального показателя — процента нарушения временного регламента экстренных служб — представлено на рисунке 2.16.

Доля выездов с нарушением регламента (> 10 мин)

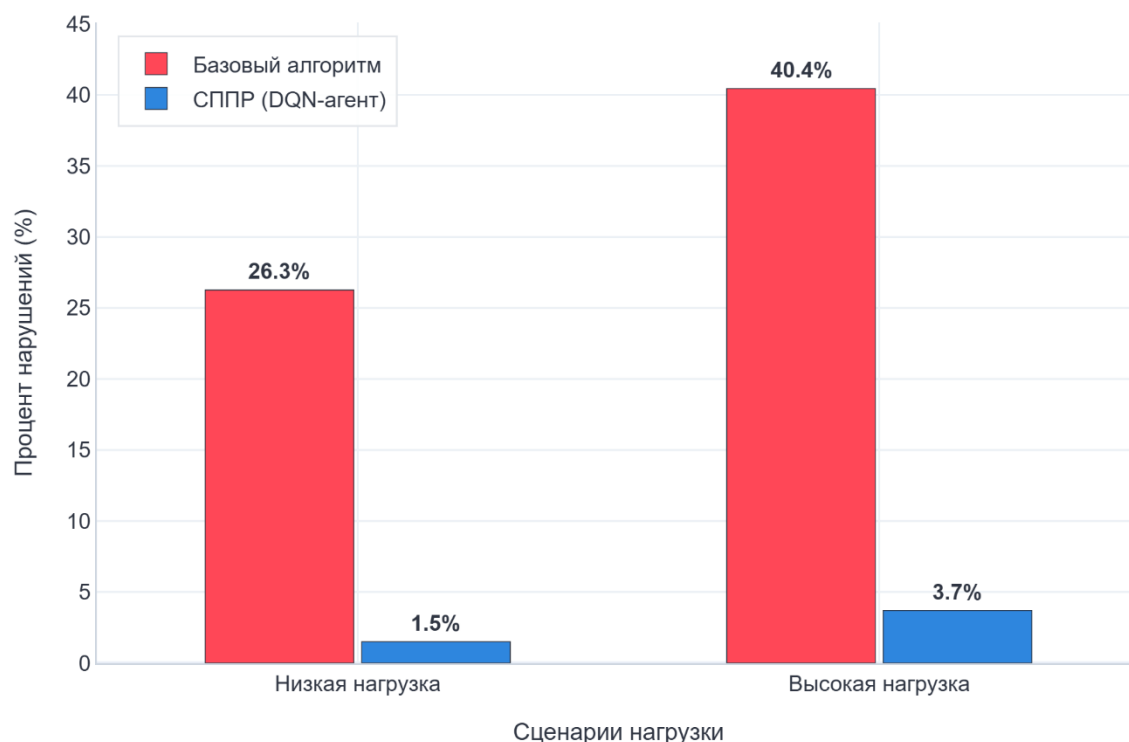


Рисунок 2.16 — Гистограмма нарушений временного регламента

Гистограмма нарушений (рисунок 2.16) подводит главный итог экспериментального сравнения систем диспетчеризации. Жадный алгоритм реагирования, ориентирующийся исключительно на евклидову метрику «слепых» расстояний без учета гетерогенности техники, демонстрирует экспоненциальный рост кол-во нарушений при увеличении нагрузки (с 26.26% до 40.44%).

2.10 Выводы

1. В рамках выполнения работы спроектирована и программно реализована масштабируемая двухуровневая архитектура системы поддержки принятия решений. Архитектура объединяет имитационную ГИС-среду AnyLogic и вычислительное ядро Python/PyTorch, синхронизированные посредством бестаймаутного сетевого шлюза Pipeline.

2. Детально проработана математическая структура нейросетевого агента (DQN). Спроектирована функция векторизации сложных пространственных и гетерогенных состояний в плоские нормализованные тензоры. Определена логика расчета функции штрафов, мотивирующая агента к минимизации задержек и экономии дефицитных видов техники.

3. Разработаны эргономичные пользовательские интерфейсы двух уровней доступа: рекомендательное рабочее место оперативного диспетчера с динамической картой маршрутов и панель системного администратора для тонкой настройки, мониторинга метрик обучения (Loss, Reward) и управления весовыми коэффициентами модели.

4. Процесс тестового обучения подтвердил математическую состоятельность выбранного подхода: алгоритм устойчиво сходится, демонстрируя переход от случайных «жадных» действий к выверенной стратегии распределения дефицитных ресурсов экстренных служб. Сформулированы критерии оценки для последующего проведения вычислительных экспериментов.

5. Проведенные экспериментальные исследования на базе репрезентативной выборки в 6000 испытаний для каждого из 4 запусков при строгом контроле воспроизводимости среды доказали статистически значимое превосходство разработанной нейросетевой модели над классическим пространственным жадным алгоритмом.

6. Обоснована методология фиксации стохастических факторов: использование независимых генераторов начальных чисел ($seed = 1$ в AnyLogic

и функция `seed_everything` в Python) обеспечило стопроцентную воспроизводимость результатов и чистоту сравнения.

7. Подтверждена гипотеза об успешном формировании неявного «знания местности» у DQN-агента. Получая на вход одинаковую с базовым алгоритмом матрицу евклидовых расстояний, нейросеть в процессе обучения перестраивает восприятие этих данных, соотнося их с реальным временем движения по дорожному графу и штрафами за пробки.

8. В штатном режиме функционирования (Сценарий №1) применение нейросетевой СППР позволило снизить среднее время реагирования на 50.88% (с 6.27 до 3.08 минут), минимизировав долю вызовов с нарушением временного регламента до рекордных 1.51%.

9. В жестких условиях высокой интенсивности пожаров и загруженности дорог (Сценарий №2) разработанный алгоритм сократил среднее время доставки техники на 61.98% (с 12.68 до 4.82 минут). При этом дисперсия времени реагирования снизилась почти в 10 раз (с 117.60 до 11.95), что свидетельствует о высокой стабильности предложенного подхода.

10. На основе сравнительного анализа диаграмм размаха установлено, что DQN-агент практически полностью поглощает экстремальные выбросы времени прибытия (свыше 25 минут), которые у базового алгоритма носят регулярный характер. Это достигается за счет предиктивного резервирования дефицитной высотной техники (составляющей 20% парка) и динамического обхода транспортных заторов.

ЗАКЛЮЧЕНИЕ

В ходе выполнения выпускной квалификационной работы была успешно достигнута поставленная цель — разработана нейросетевая модель и комплекс программных средств для повышения эффективности маршрутизации и диспетчеризации пожарной автотехники в условиях городской среды.

В процессе работы были решены все сформулированные задачи и получены следующие основные теоретические и практические результаты:

Проведен системный анализ предметной области и существующих методов маршрутизации. Выявлено, что главной проблемой современных автоматизированных систем экстренного реагирования является применение детерминированных «жадных» алгоритмов. Показано, что такие алгоритмы решают лишь локальную задачу поиска кратчайшего пути, но не способны предиктивно управлять структурным дефицитом специализированной пожарной техники (в частности, автолестниц, доля которых составляет лишь около 20% от общего оперативного автопарка).

Спроектирована и реализована оригинальная двухуровневая архитектура программного комплекса. Для преодоления вычислительных ограничений классических симуляторов система была разделена на исполнительную среду (AnyLogic / Java) и интеллектуальное ядро (Python / PyTorch). Надежность межпроцессного взаимодействия и строгая синхронизация виртуального времени были обеспечены за счет внедрения бестаймаутного сокетного шлюза Pipeline.

Разработана высокодетализированная имитационная модель. В среде AnyLogic воссоздан участок урбанизированной территории на базе реальных ГИС-данных OpenStreetMap, реализован ориентированный дорожный граф, стохастический генератор инцидентов различного ранга сложности и алгоритмы навигации гетерогенной популяции агентов (автоцистерн и автолестниц).

Сформирована и обучена глубокая Q-сеть (DQN). Процесс управления распределением техники был математически формализован как Марковский процесс принятия решений (MDP). Нейросетевой агент, используя многомерные векторы состояний и буфер воспроизведения опыта (Experience Replay), успешно обучился стратегиям минимизации интегрального времени реагирования. Реализована функция отрицательного вознаграждения (штрафа), пропорциональная фактическому времени нахождения техники в пути. Обучение глубокой Q-сети оптимизировано под концепцию асинхронных терминальных эпизодов, что позволило математически строго редуцировать уравнение оптимальности Беллмана к оценке немедленного отклика среды,

снизить вычислительную сложность алгоритма в режиме реального времени и исключить некорректные временные корреляции между параллельно развивающимися инцидентами.

Спроектированы эргономичные пользовательские интерфейсы двух уровней. Создано специализированное рабочее место оперативного диспетчера с динамической ГИС-картой и рекомендательной ИИ-панелью (AI Overlay), поддерживающее концепцию ручного перехвата управления (Human-in-the-Loop). Разработана панель администратора для мониторинга метрик сходимости, стресс-тестирования среды и обновления весов нейронной сети.

Проведено экспериментальное исследование эффективности системы. Сравнительный анализ DQN-модели и базового жадного алгоритма подтвердил математическую и практическую состоятельность разработанной СППР. При пиковых стрессовых нагрузках на инфраструктуру алгоритм на базе искусственного интеллекта продемонстрировал способность к стратегическому «ресурсному резервированию».

Количественные итоги исследования:

В условиях штатного режима функционирования применение нейросетевой модели позволило сократить математическое ожидание времени реагирования в 2,03 раза (с 6,27 до 3,08 минут) при снижении доли вызовов с нарушением 10-минутного нормативного регламента МЧС [9] с 26,26% до 1,51%. В жестких условиях стрессовой нагрузки, где базовый алгоритм показал системный сбой (среднее время прибытия 12,68 минут, доля нарушений — 40,44%), интеллектуальная СППР успешно удержала временные показатели в безопасном диапазоне, снизив среднее время доставки на 61,98% (до 4,82 минут), а уровень нарушений — до 3,70%.

Доля нарушений нормативного регламента прибытия экстренных служб, установленного Федеральным законом № 123-ФЗ [9] (регламентный лимит в 10 минут для городских поселений), в условиях штатной нагрузки снизилась с 26,26% до 1,51%, а в режиме пикового стохастического потока инцидентов — с критических 40,44% до безопасных 3,70%.

Таким образом, разработанный программный комплекс представляет собой полнофункциональный прототип интеллектуальной системы поддержки принятия решений (СППР) диспетчера. Практическая значимость работы заключается в возможности интеграции предложенных алгоритмов глубокого обучения с подкреплением и разработанных пользовательских интерфейсов в действующие автоматизированные контуры Центров управления силами (ЦУС) ФПС ГПС МЧС России, что позволит существенно повысить надежность, адаптивность и оперативность реагирования пожарно-спасательных подразделений в условиях динамической среды современных мегаполисов.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Беллман, Р. Динамическое программирование / Р. Беллман ; пер. с англ. И. М. Сироты. – Москва: Издательство иностранной литературы, 1960. – 400 с.
2. Борщев, А. В. Практическое агентное моделирование и его место в арсенале аналитика / А. В. Борщев // Exponenta Pro. Математика в приложениях. – 2004. – № 3-4 (7-8). – С. 38–47.
3. Брушлинский, Н. Н. Системный анализ и проблемы пожарной безопасности / Н. Н. Брушлинский, С. В. Соколов, П. Вагнер. – Москва : Стройиздат, 1988. – 413 с.
4. Карпов, Ю. Г. Имитационное моделирование систем. Введение в моделирование с AnyLogic 5 / Ю. Г. Карпов. – Санкт-Петербург: БХВ-Петербург, 2005. – 400 с. – ISBN 5-94157-148-8.
5. Карташев, О. А. Использование геоинформационных систем и интеллектуальных алгоритмов для оптимизации маршрутов движения пожарной техники / О. А. Карташев // Пожаровзрывобезопасность. – 2022. – Т. 31, № 4. – С. 45–53. – Текст: непосредственный.
6. Кормен, Т. Алгоритмы: построение и анализ / Т. Кормен, Ч. Лейзерсон, Р. Ривест, К. Штайн ; пер. с англ. – 3-е изд. – Москва : Вильямс, 2013. – 1328 с. – ISBN 978-5-8459-1794-8.
7. Рассел, С. Искусственный интеллект: современный подход / С. Рассел, П. Норвиг ; пер. с англ. и ред. К. А. Птицына. – 3-е изд. – Москва: Вильямс, 2015. – 1408 с. – ISBN 978-5-8459-1504-3.
8. Саттон, Р. С. Обучение с подкреплением / Р. С. Саттон, Э. Г. Барто ; пер. с англ. А. А. Слинкина. – 2-е изд. – Москва: ДМК Пресс, 2020. – 618 с. – ISBN 978-5-97060-097-9.
9. Федеральный закон от 22 июля 2008 г. № 123-ФЗ (ред. от 14 июля 2022 г.) «Технический регламент о требованиях пожарной безопасности» : (с изм. и доп., вступ. в силу с 01.03.2025) // КонсультантПлюс : [справочно-правовая система]. – URL:

http://www.consultant.ru/document/cons_doc_LAW_78699/ (дата обращения: 15.06.2026). – Текст : электронный.

10. AnyLogic: Имитационное моделирование для бизнеса: [официальный сайт]. – URL: <https://www.anylogic.ru> (дата обращения: 15.06.2026). – Текст: электронный.

11. Mnih, V. Human-level control through deep reinforcement learning / V. Mnih, K. Kavukcuoglu, D. Silver [et al.] // Nature. – 2015. – Vol. 518, № 7540. – P. 529–533. – DOI 10.1038/nature14236.

12. OpenStreetMap: [свободная географическая карта мира]. – URL: <https://www.openstreetmap.org> (дата обращения: 15.06.2026). – Текст: электронный.

13. Paszke, A. PyTorch: An Imperative Style, High-Performance Deep Learning Library / A. Paszke, S. Gross, F. Massa [et al.] // Advances in Neural Information Processing Systems 32. – Curran Associates, Inc., 2019. – P. 8024–8035.

14. Pypeline: Custom Python integration for AnyLogic models / AnyLogic // GitHub: [платформа для хостинга ИТ-проектов]. – URL: <https://github.com/AnyLogic/pypeline> (дата обращения: 15.06.2026). – Текст : электронный.

15. PyTorch Documentation: [официальный сайт]. – URL: <https://pytorch.org/docs/stable/index.html> (дата обращения: 15.06.2026). – Текст: электронный.

16. AnyLogic 8. Руководство пользователя: [официальный сайт] / ООО «Компания ЭниЛоджик». – 2026. – URL: <https://help.anylogic.ru> (дата обращения: 15.06.2026). – Текст: электронный.

ПРИЛОЖЕНИЕ А

Министерство науки и высшего образования Российской Федерации
Мытищинский филиал федерального государственного автономного образовательного учреждения высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МФ МГТУ им. Н.Э. Баумана)



Факультет «Космический»
Кафедра «Прикладная математика, информатика и вычислительная техника»
Направление подготовки 09.03.01 - «Информатика и вычислительная техника»

Выпускная квалификационная работа на тему:

**«Разработка нейросетевой модели для поддержки принятия решения при маршрутизации
пожарной автотехники в городской среде »**

Выполнил: Нистратов Радимир Алексеевич
Руководитель : Вилисов Валерий Яковлевич

Задачи

01

Анализ систем маршрутизации

Исследование факторов принятия решений диспетчером пожарной техники

02

Анализ RL алгоритмов

Применение обучения с подкреплением в задачах экстренного реагирования

03

Математическая постановка

Формализация задачи (MDP) и проектирование архитектуры нейросетевой СППР

04

Имитационное моделирование

Разработка модели города в AnyLogic со случайными заторами и инцидентами

05

Программная интеграция

Связь среды AnyLogic и Python через Pyreline для работы нейросетевого модуля

06

Интерфейсы UI/UX

Создание панелей управления для диспетчера и администратора системы

07. Экспериментальное исследование и оценка эффективности

Сравнительный анализ работы СППР с базовыми алгоритмами маршрутизации

Основные задачи и требования к системам маршрутизации в пожарной службе

Законодательное ограничение

Согласно ФЗ № 123-ФЗ, регламентированное время прибытия первого пожарного подразделения в городе не должно превышать 10 минут

Основные оперативные задачи СППР:

01

Автоматическая верификация адреса и определение ранга сложности пожара

02

Идентификация доступных сил в ближайших гарнизонах

03

Расчет оптимальной траектории движения с учетом дорожных ограничений

3

Критерии эффективности и ограничения

Главный критерий

Минимизация интегрального времени реагирования

$$T_{\text{реар}} \rightarrow \min$$

Вторичный критерий

Минимизация пройденного расстояния

$$S \rightarrow \min$$

4

Анализ существующих методов поддержки принятия решений

Классические детерминированные подходы

Методы:

Алгоритмы Дейкстры, A*, Флойда -Уоршелла, методы решения задач VRP

Плюсы:

Математическая строгость и гарантированная сходимость точного оптимального пути на статическом графе

Минусы:

Не способны адаптироваться к изменениям среды и распределять дефицитные ресурсы

Интеллектуальные (нейросетевые) методы

Методы:

Глубокое обучение с подкреплением (DRL), в частности алгоритмы DQN

Плюсы:

Высокая скорость, учет пространственно -временных закономерностей города, предиктивное резервирование ресурсов

Минусы:

Требуют длительного и ресурсоемкого процесса предварительной тренировки агента

5

Архитектура системы

Двухуровневая распределенная архитектура: Создана для устранения конфликта между высокой детализацией симуляции и скоростью обучения нейросети.

Исполнительный уровень

AnyLogic / Java

Микроскопическое агентное моделирование физических процессов, навигация, сбор телеметрии

Интеграционный уровень

шлюз Pipeline

Бестаймаутный сокет-сервер для обмена данными по протоколу TCP/IP

Ядро интеллекта

Python / PyTorch

Вычислительный модуль аппроксимации функций ценности действий

Пространственная среда

ГИС-данные OpenStreetMap (OSM) в виде ориентированного графа с 4 депо. Внедрены зоны заторов (**jamZone**), агент обучается в условиях частичной наблюдаемости

6

Теоретический базис: Марковский процесс принятия решений (MDP)

Математическая формализация задачи: Процесс последовательного выбора оптимальных диспетчерских решений в стохастической городской среде формализован как Марковский процесс принятия решений.

Базовая структура процесса: $MDP = (S, A, P, R, \gamma)$, где:

S — конечное множество многомерных состояний среды;

A — дискретное пространство допустимых управляющих воздействий;

P — стохастическая матрица вероятностей переходов между состояниями;

R — скалярная функция немедленного вознаграждения(штрафа);

γ — коэффициент дисконтирования.

7

Реализация марковского процесса принятия решений в СППР: Состояние, Действие и Награда

Формализация состояния среды (S_t): Многомерная ресурсная и пространственная информация AnyLogic преобразуется в плоский нормализованный вектор:

- Категорию сложности текущего инцидента (`severity`)
- Вектор евклидовых расстояний от очага пожара до каждого из 4 -х депо
- Матрицу остатка свободных ресурсов индивидуально по каждой станции (`station_status`)

Формализация пространства действий (A_t): Выбор конкретного номера (индекса) пожарной станции для отправки боевых расчетов на верифицированный инцидент

Расчет функции вознаграждения (R): На основе фактического времени в пути:

$$R = -T_{\text{пути}} \cdot n,$$

где R — скалярный показатель отклика (значение вознаграждения или штрафа) виртуальной среды ;

$T_{\text{пути}}$ — фактическое время пожарной техники в пути, полученное от модели симуляции ;

n — коэффициент скорости обучения ($n = 10$).

8

Структура передаваемых данных через шлюз Pipeline

Вектор состояния (s)

```
{
  "fire_id": 1042,
  "severity": 1,
  "distances": [42, 127, 70, 321],
  "station_status": [
    [2, 1],
    [0, 0],
    [2, 1],
    [1, 0]
  ]
}
```

Вектор действий (a)

```
{
  "incident_id": 1042,
  "selected_action": 1
}
```

9

Протокол сопряжения и синхронизация времени

Транспортное обеспечение: Межпроцессное взаимодействие (IPC) на loopback (TCP/IP)

Проблема времени: Предотвращение искусственной задержки выезда техники за счет остановки виртуального времени AnyLogic во время работы нейросети

Событийно-блокирующий алгоритм синхронизации

01. Пауза

Заморозка симуляции при новом инциденте:

```
getEngine().pause();
```

02. Экспорт

Формирование пакета состояния и отправка через шлюз Pipeline по TCP/IP

03. Расчет

Python обрабатывает JSON, выполняет forward-проход DQN и возвращает действие

04. Возобновление

Прием команды, отправка техники и разморозка:

```
getEngine().resume();
```

10

Архитектура нейросети

Глубокая Q-сеть (DQN)

Полносвязный перцептрон (MLP):

- **Входной слой:** Размерность вектора состояния S_t
- **Скрытые слои:** 2 слоя по 64 нейрона с активацией ReLU
- **Выходной слой:** n нейронов (Q-значения для станций)
- **Оптимизатор:** Adam (LR=0.001), функция потерь MSE

Механизм обучения

- **Replay Buffer:** База данных из 5000 последних переходов для устранения автокорреляции
- **ϵ -greedy:** Баланс Exploration/Exploitation (decay 0.995)
- **Batch Training:** Обучение пакетами по 32 примера

11

Реализация метода обучения

Обновление весов

Вычисление критерия оптимизации и обновление весов нейросети автоматизированы в [PyTorch](#)

Используются стандартные высокоуровневые абстракции для эффективного градиентного спуска

```
def train_model():
    global current_loss, current_reward
    if len(memory) < BATCH_SIZE:
        return
    batch = random.sample(memory, BATCH_SIZE)
    states, actions, rewards = zip(*batch)
    states = torch.FloatTensor(np.array(states))
    actions = torch.LongTensor(actions).unsqueeze(1)
    rewards = torch.FloatTensor(rewards).unsqueeze(1)
    q_values = model(states).gather(1, actions)
    loss = nn.MSELoss()(q_values, rewards)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
    current_loss = loss.item()
    current_reward = rewards.mean().item()
    print(f" [RL] Train Step: Loss: {current_loss:.4f} | Avg Reward: {current_reward:.2f} | Epsilon: {EPSILON:.2f}")
```

Функция потерь (MSE)

Оценка расхождения выполняется с помощью встроенного класса `nn.MSELoss()`

Модуль автоматически рассчитывает отклонение между:

- Вектором текущих оценок полезности (выход Q-сети)
- Вектором временных целевых ориентиров (Target Q)

Это обеспечивает стабильную сходимость алгоритма DQN

12

Гетерогенность ресурсов автопарка

fireTrucks - FireTruck

Имя: fireTrucks ☒ Отображать имя ☐ Исключить

☐ Одиночный агент ☒ Популяция агентов

Популяция: ☐ Изначально пуста ☒ Содержит заданное кол-во агентов ☐ Загружается из базы данных

Начальное количество агентов: 5

type: randomTrue(0.8) ? TANKER : LADDER

homeBase: randomFrom(allStations)

working: ☐

Настройка генерации гетерогенной популяции и визуализации машин в AnyLogic

80% ПАРКА

Автоцистерна (TANKER)

Визуализация: Красный цвет (Red)

Базовый ресурс. Направляется на инциденты низкого ранга сложности

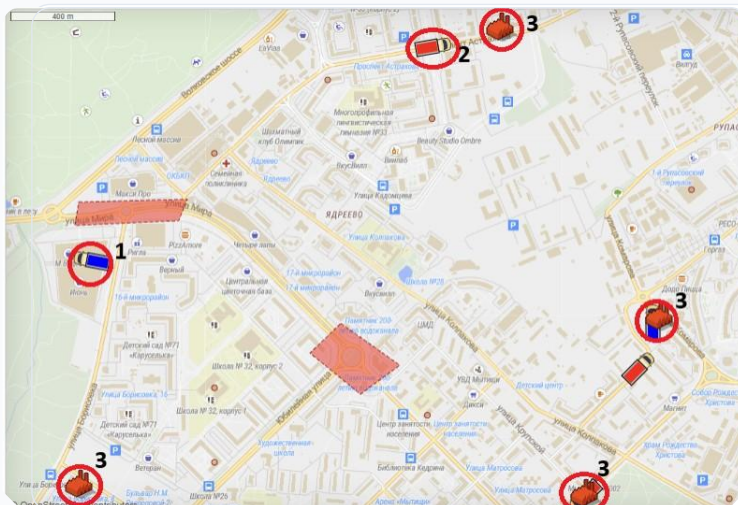
20% ПАРКА

Автолестница (LADDER)

Визуализация: Синий цвет (Blue)

Дефицитный ресурс. Критически необходим для высотных инцидентов

Объекты на карте имитационной модели



Условные обозначения

1

Автолестница

Специализированная техника

2

Автоцистерна

Базовая пожарная техника

3

Пожарные станции

Места дислокации экипажей

Роли пользователей и интерфейсы

Диспетчер (ЛПР)

Лицо, принимающее решения в оперативной обстановке. Основной пользователь системы поддержки принятия решений (СППР)

- Контроль инцидентов в реальном времени
- Подтверждение предложенных алгоритмом решений

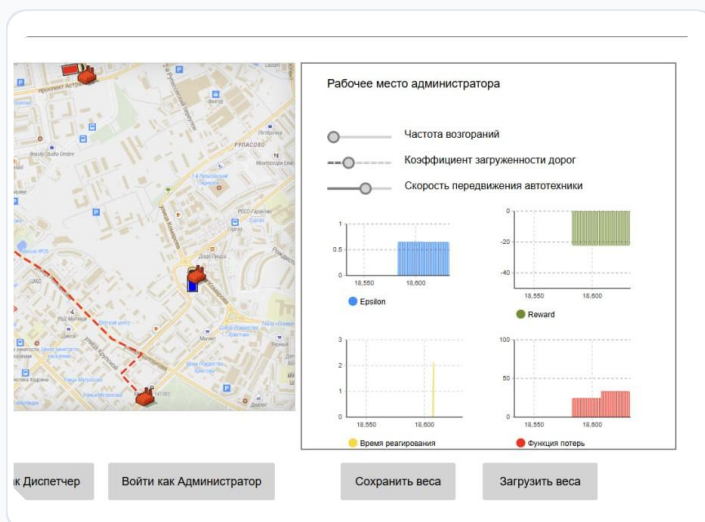
Администратор

Технический специалист, ответственный за конфигурацию системы и обучение моделей машинного обучения

- Управление параметрами имитационной среды
- Дообучение и сохранение весов нейронной сети

15

Интерфейс администратора



Ключевые функциональные возможности:

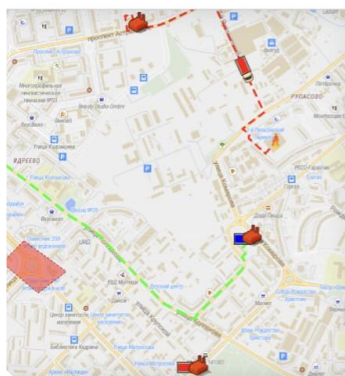
- Настройка параметров среды
- Мониторинг графиков обучения
- Сохранение/Загрузка весов модели

16

Интерфейс диспетчера

Ключевые функциональные возможности:

- Мониторинг времени реагирования и анимация маршрутов
- Управление СППР и рекомендации от DQN-модели
- Контроль резерва техники (АЦ/АП) по всем депо
- Выбор режима работы программы



17

Методология и условия проведения экспериментов

Цель исследований:

Сравнительная оценка эффективности разработанной СППР на базе алгоритма DQN и классических диспетчерских эвристик в ГИС-среде города

Обеспечение чистоты эксперимента:

В среде AnyLogic:

Фиксация генератора случайных чисел гарантирует абсолютно идентичный поток, координаты и сложность пожаров во всех тестах

В среде Python:

Комплексная функция `seed_everything(seed = 1)` для полной детерминированности весов сети, буфера памяти и выбора действий

Объем выборки и равенство условий:

- Собрано по **6000** последовательных измерений для каждого запуска
- Оба алгоритма получают на вход одинаковую изначальную матрицу евклидовых расстояний
- Состав автопарка зафиксирован: **80%** автоцистерн (TANKER) и **20%** автолестниц (LADDER)

18

Сценарии моделирования

Инструмент управления:

Параметры внешней среды регулируются с помощью трех интерактивных слайдеров:

- Вероятность пожара (p): от 0 до 1
- Загруженность сети (k): от 1 до 10
- Скорость техники (v): от 1 до 100 у.е.

Сценарий № 1

(Штатный режим)

Умеренная интенсивность инцидентов и относительно свободная дорожная сеть

Параметры:

$p = 0.2$, $k = 4$, $v = 60$

Сценарий № 2

(Экстремальные условия)

Имитация часов пик, неблагоприятной погоды и резкого роста числа ЧС

Параметры:

$p = 0.6$, $k = 8$, $v = 40$

19

Описание сравниваемых алгоритмов

1. Базовый алгоритм (ближайший по радиусу)

Реализует классический «жадный» пространственный поиск с использованием ГИС-функции для расчета расстояния по прямой

Направляет экипаж из ближайшего депо с любой свободной машиной

Критический недостаток: Алгоритм «слеп» к типам машин (`hasFreeTruck = true`). Это ведет к растрате дефицитных автолестниц на простые вызовы

2. Нейросетевой алгоритм (DQN-агент)

Принимает решения на базе вектора состояний с матрицей расстояний и учетом типов машин (`stationStatus`)

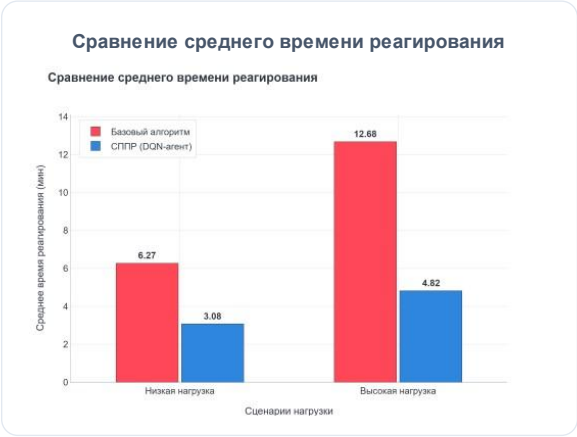
Соотносит евклидовы расстояния с реальным временем движения, адаптируясь к зонам заторов (`jamZones`)

Учится стратегически «придерживать» автолестницы для сложных высотных вызовов (Ранг 2)

20

Анализ результатов вычислительных экспериментов

Среднее время реагирования у СППР DQN меньше на ~50% при низкой нагрузке. При высокой- на ~62%



21

Результаты моделирования: Сценарий № 1 (Низкая нагрузка)

Плотность распределения СППР DQN имеет ярко выраженную правостороннюю асимметрию с острым пиком в безопасном диапазоне 2.0 - 3.5 минут

Распределение «жадного» алгоритма сильно размыто по оси времени и имеет длинное правое плечо, уходящее в зону задержек

Ключевые показатели

90-й перцентиль прибытия:

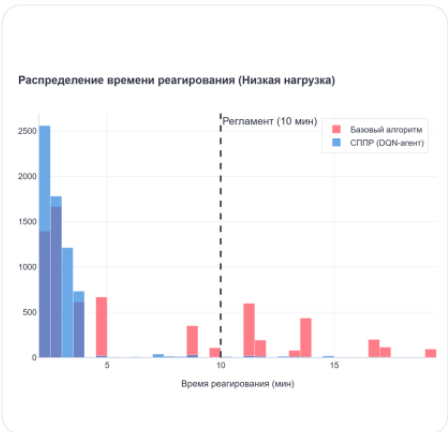
Глубокая Q-сеть (DQN): 4.0 мин

Базовый: 13.6 мин

Соблюдение регламента:

Интегральная вероятность (<10 мин) для глубокой Q-сети (DQN): **98.49%**

Доля нарушений: всего 1.51%



22

Результаты моделирования: Сценарий № 2 (Высокая нагрузка)

Плотность распределения «жадного» алгоритма полностью уплощается и приобретает тяжелый хвост вплоть до максимума в **38.14** мин. Время прибытия становится непредсказуемой величиной. СППР DQN эффективно противодействует стохастическим заторам, сохраняя плотный пик распределения в пределах **5 минут**

Ключевые показатели

90-й перцентиль прибытия:

Глубокая Q-сеть (DQN): 5.2 мин

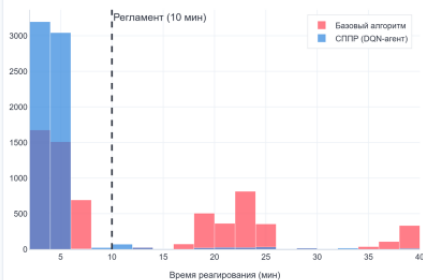
Базовый: 25 мин

Соблюдение регламента:

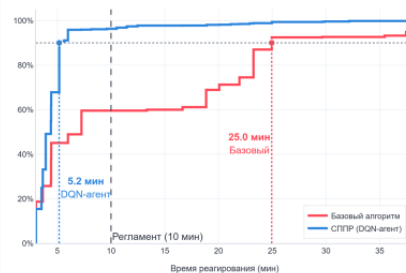
Вероятность (<10 мин) для глубокой Q-сети (DQN): **96.7%**

Доля нарушений: всего 3.7%

Распределение времени реагирования (Высокая нагрузка)



Кумулятивное распределение времени реагирования (Высокая нагрузка)



23

Статистическая стабильность и анализ размаха данных

Межквартильный размах (IQR)

СППР (DQN-агент): жестко сжат в узких пределах:

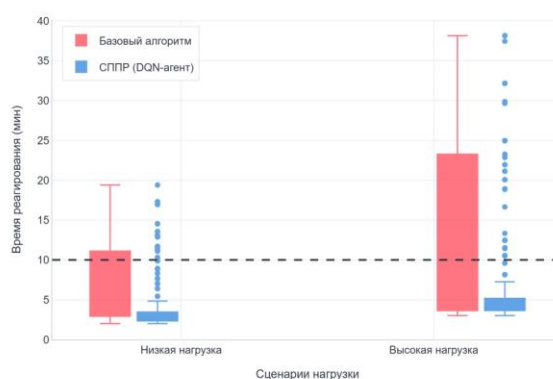
- Низкая нагрузка: [2.2, 3.4] мин
- Высокая нагрузка: [3.2, 5.0] мин

Это доказывает высокую однородность и стабильность решений нейросети

Базовый алгоритм:

Тело ящика значительно шире, а верхний ус при высокой нагрузке уходит далеко за предел 20 минут

Сравнение размаха и стабильности времени реагирования



24

Результаты работы

Спроектированы два интерфейса

Реализованы рабочее место диспетчера и администратора

Обучена модель DQN

На базе MDP реализована стратегия предиктивного резервирования дефицитных ресурсов

Разработан комплекс СППР

Создана распределенная архитектура, объединяющая среду и нейросетевое ядро через шлюз Pipeline

25

Результаты работы (продолжение)

Эффективность системы

Время реагирования

Среднее время реагирования СППР лучше базового алгоритма:

В штатном режиме на ~50%, при высокой нагрузке на ~62%

Доля нарушений регламента (<10 мин)

Базовый алгоритм имеет 26% при низкой нагрузке и ~40% при высокой

СППР же 1,51% и 3,70% соответственно

26

Спасибо за внимание

ПРИЛОЖЕНИЕ Б

УТВЕРЖДАЮ

Зав. кафедрой КЗ-МФ

_____ А.В.Корольков

«__» _____ 2026 г.

Нейросетевая модель для поддержки принятия решения при маршрутизации
пожарной автотехники в городской среде

Спецификация

ЛИСТ УТВЕРЖДЕНИЯ

643.МГУЛ.090301-80-ЛУ

Студент

_____ Р.А. Нистратов

«__» _____ 2026 г.

Руководитель

_____ В.Я. Вилисов

«__» _____ 2026 г.

Нормоконтролер

_____ А.В. Чернышов

«__» _____ 2026 г.

УТВЕРЖДЕН

643.МГУЛ.090301-80-ЛУ

Нейросетевая модель для поддержки принятия решения при маршрутизации
пожарной автотехники в городской среде

Спецификация

643.МГУЛ.090301-80

Листов 3

643.МГУЛ.090301-80

Обозначение	Наименование	Примечание
	<u>Документация</u>	
643.МГУЛ.090301-80-ЛУ	Нейросетевая модель	Размножать
	для поддержки принятия решения	по
	при маршрутизации	указанию
	пожарной автотехники	
	в городской среде	
	Лист утверждения	
643.МГУЛ.090301-80 12 01	Нейросетевая модель	
	для поддержки принятия решения	
	при маршрутизации	
	пожарной автотехники	
	в городской среде	
	Текст программы	

643.МГУЛ.090301-80

643.МГУЛ.090301-80 12 01-ЛУ	Нейросетевая модель	Размножать
	для поддержки принятия решения	по
	при маршрутизации	указанию
	пожарной автотехники	
	в городской среде	
	Текст программы	
	Лист утверждения	
643.МГУЛ.090301-80 34 01	Нейросетевая модель	
	для поддержки принятия решения	
	при маршрутизации	
	пожарной автотехники	
	в городской среде	
	Руководство оператора	
643.МГУЛ.090301-80 34 01-ЛУ	Нейросетевая модель	Размножать
	для поддержки принятия решения	по
	при маршрутизации	указанию
	пожарной автотехники	
	в городской среде	
	Руководство оператора	
	Лист утверждения	

УТВЕРЖДАЮ

Зав. кафедрой К-3 МФ

_____ А.В.Корольков.

«__» _____ 2026 г.

Нейросетевая модель для поддержки принятия решения при маршрутизации
пожарной автотехники в городской среде

ЛИСТ УТВЕРЖДЕНИЯ

643.МГУЛ.090301-80 12 01-ЛУ

Студент

_____ Р. А. Нистратов

«__» _____ 2026 г.

Руководитель

_____ В. Я. Вилисов

«__» _____ 2026 г.

Нормоконтролер

_____ А.В. Чернышов

«__» _____ 2026 г.

УТВЕРЖДЕН

643.МГУЛ.090301-80 12 01-ЛУ

Нейросетевая модель для поддержки принятия решения при маршрутизации
пожарной автотехники в городской среде

Текст программы

643.МГУЛ.090301-80 12 01

Листов 6


```
import torch

import torch.nn as nn

import torch.optim as optim

import numpy as np

import random

from collections import deque

import os


def seed_everything(seed=42):

    random.seed(seed)

    os.environ['PYTHONHASHSEED'] = str(seed)

    np.random.seed(seed)

    torch.manual_seed(seed)

    if torch.cuda.is_available():

        torch.cuda.manual_seed(seed)

        torch.cuda.manual_seed_all(seed)

    torch.backends.cudnn.deterministic = True

    torch.backends.cudnn.benchmark = False


seed_everything(42)


LR = 0.001

BATCH_SIZE = 32

EPSILON = 1.0

EPSILON_MIN = 0.05
```

```
EPSILON_DECAY = 0.995
```

```
current_loss = 0.0
```

```
current_reward = 0.0
```

```
memory = deque(maxlen=5000)
```

```
active_transitions = {}
```

```
class DQN(nn.Module):
```

```
    def __init__(self, input_dim, output_dim):
```

```
        super(DQN, self).__init__()
```

```
        self.fc1 = nn.Linear(input_dim, 64)
```

```
        self.fc2 = nn.Linear(64, 64)
```

```
        self.fc3 = nn.Linear(64, output_dim)
```

```
    def forward(self, x):
```

```
        x = torch.relu(self.fc1(x))
```

```
        x = torch.relu(self.fc2(x))
```

```
        return self.fc3(x)
```

```
model = None
```

```
optimizer = None
```

```
def flatten_state(severity, distances, station_status):
```

```
    state = [severity]
```

```
    state.extend(distances)
```

```
for status in station_status:
```

```
    state.extend(status)
```

```
return np.array(state, dtype=np.float32)
```

```
def get_action(fire_id, severity, distances, station_status):
```

```
    global model, optimizer, active_transitions
```

```
    state = flatten_state(severity, distances, station_status)
```

```
    input_dim = len(state)
```

```
    output_dim = len(distances)
```

```
    if model is None:
```

```
        model = DQN(input_dim, output_dim)
```

```
        optimizer = optim.Adam(model.parameters(), lr=LR)
```

```
        print(f"\n[RL] Model created. Inputs: {input_dim}, Outputs: {output_dim}")
```

```
    if random.random() < EPSILON:
```

```
        action = random.randint(0, output_dim - 1)
```

```
    else:
```

```
        state_tensor = torch.FloatTensor(state).unsqueeze(0)
```

```
        with torch.no_grad():
```

```
            q_values = model(state_tensor)
```

```
        action = torch.argmax(q_values).item()
```

```
    active_transitions[fire_id] = {'state': state, 'action': action}
```

```
return action
```

```
def update_reward(fire_id, travel_time):
```

```
    global active_transitions, EPSILON, current_loss
```

```
    reward = -travel_time * 10.0
```

```
    # with open("dqn_high.csv", "a", encoding="utf-8") as log_file:
```

```
    #     log_file.write(f'{fire_id},{travel_time}\n')
```

```
    if fire_id in active_transitions:
```

```
        transition = active_transitions.pop(fire_id)
```

```
        memory.append((transition['state'], transition['action'], reward))
```

```
        print(f'    [PYTHON] Memory updated for fire {fire_id}. Buffer:
{len(memory)}/{memory.maxlen}')
```

```
        if EPSILON > EPSILON_MIN:
```

```
            EPSILON *= EPSILON_DECAY
```

```
        train_model()
```

```
def train_model():
```

```
    global current_loss, current_reward
```

```
    if len(memory) < BATCH_SIZE:
```

```
        return
```

```
    batch = random.sample(memory, BATCH_SIZE)
```

```
    states, actions, rewards = zip(*batch)
```

```

states = torch.FloatTensor(np.array(states))
actions = torch.LongTensor(actions).unsqueeze(1)
rewards = torch.FloatTensor(rewards).unsqueeze(1)

q_values = model(states).gather(1, actions)

loss = nn.MSELoss()(q_values, rewards)

optimizer.zero_grad()
loss.backward()
optimizer.step()
current_loss = loss.item()
current_reward = rewards.mean().item()

print(f" [RL] Train Step! Loss: {current_loss:.4f} | Avg Reward:
{current_reward:.2f} | Epsilon: {EPSILON:.2f}")

def save_weights():
    global model
    if model is not None:
        torch.save(model.state_dict(), 'dqn_weights.pth')
        return "Успешно: Веса нейросети сохранены (dqn_weights.pth)!"
    else:
        return "Ошибка: Модель еще не создана. Сгенерируйте хотя бы один
пожар!"

```

```
def load_weights():
    global model, EPSILON
    if model is not None:
        if os.path.exists('dqn_weights.pth'):
            try:
                model.load_state_dict(torch.load('dqn_weights.pth'))
                EPSILON = EPSILON_MIN
                return "Успешно: Обученные веса загружены! Epsilon снижен."
            except Exception as e:
                return f"Ошибка при загрузке: {e}"
        else:
            return "Ошибка: Файл dqn_weights.pth не найден. Сначала сохраните веса."
    else:
        return "Ошибка: Модель еще не инициализирована. Сгенерируйте хотя бы один пожар перед загрузкой!"

def get_metrics():
    global current_loss, EPSILON, current_reward
    return f"{current_loss},{EPSILON},{current_reward}"
```

[illegible]

УТВЕРЖДАЮ

Зав. кафедрой К-3 МФ

_____ А.В.Корольков.

«__»_____ 2026 г.

Нейросетевая модель для поддержки принятия решения при маршрутизации
пожарной автотехники в городской среде

Руководство оператора

ЛИСТ УТВЕРЖДЕНИЯ

643.МГУЛ.090301-80 34 01-ЛУ

Студент

_____ Р. А. Нистратов

«__»_____ 2026 г.

Руководитель

_____ В. Я. Вилисов

«__»_____ 2026 г.

Нормоконтролер

_____ А.В. Чернышов

«__»_____ 2026 г.

УТВЕРЖДЕН

643.МГУЛ.090301-80 34 01-ЛУ

Нейросетевая модель для поддержки принятия решения при маршрутизации
пожарной автотехники в городской среде

Руководство оператора

643.МГУЛ.090301-80 34 01

Листов 8

АННОТАЦИЯ

В данном программном документе приведено руководство оператора к нейросетевой модели для поддержки принятия решения при маршрутизации пожарной автотехники в городской среде

СОДЕРЖАНИЕ

1	Назначение программы	4
2	Условия выполнения программы.....	4
3	Выполнение программы.....	5
4	Сообщения оператору.....	6

1 Назначение программы

Программный комплекс предназначен для имитационного моделирования и оптимизации маршрутизации пожарной автотехники (гетерогенного парка автоцистерн и автолестниц) в городской среде с использованием алгоритмов глубокого обучения с подкреплением. Программа предоставляет автоматизированные рабочие места с графическими интерфейсами для оперативного диспетчера и системного администратора.

2 Условия выполнения программы

Для корректного запуска и работы программного комплекса необходимо выполнение следующих минимальных системных и программных требований:

- Установленная среда агентного имитационного моделирования AnyLogic.
- Установленный интерпретатор языка Python.
- Установленные библиотеки машинного обучения и вычислений для Python: torch (PyTorch) и numpy.
- Наличие интегрированного сокетного шлюза Pipeline для обеспечения бесшовного межпроцессного взаимодействия и синхронизации виртуального времени между JVM (AnyLogic) и вычислительным ядром Python.

3 Выполнение программы**1) Запуск программы**

- Откройте файл имитационной модели проекта в среде AnyLogic.
- Убедитесь, что в настройках элемента Pipeline прописан корректный системный путь к установленному на вашем ПК интерпретатору Python.
- Запустите модель из интерфейса AnyLogic. Вычислительный скрипт agent.py будет запущен автоматически, TCP/IP соединение установится без дополнительных действий со стороны пользователя.

2) 3.2 Работа в режиме оперативного диспетчера (СППР)

- После старта симуляции откроется основное окно НМИ, центральным элементом которого является масштабируемая ГИС-карта.
- В левой части экрана на рекомендательной панели отслеживайте блок «Статус пожарных станций». Он показывает точное количество свободных автоцистерн (АЦ) и автолестниц (АЛ) в каждом из четырех депо.
- Для выбора степени автономности используйте радиокнопки: «Без ИИ», «Автопилот ИИ» или «СППР (Интерактивный)».
- При возникновении инцидента на карте отрисовывается специализированный маркер пожара. В интерактивном режиме ИИ-помощник выполнит предиктивный расчет и предложит оптимальную станцию в строке «Рекомендация ИИ».
- Для подтверждения выезда нажмите кнопку «[Подтвердить выбор ИИ]» или вручную назначьте другую часть кнопкой «[Отправить Депо X]».
- Маршрут следования техники динамически отображается на карте: красным цветом визуализируются базовые АЦ, синим — дефицитные АЛ.

3) 3.3 Работа в панели администратора (настройка и обучение)

- Переключитесь на интерфейс службы поддержки и администрирования модели.

- В блоке управления стресс-сценариями используйте ползунки для изменения параметров в реальном времени: частота возгораний, коэффициент загруженности дорог, скорость передвижения автотехники.
- В блоке телеметрии оценивайте качество работы агента по динамическим графикам: Функция потерь (Loss Function), Вознаграждение (Reward), Время реагирования и параметр исследования Epsilon.
- Для сохранения текущих параметров обученной нейросети (для резервного копирования или фиксации удачного эксперимента) нажмите кнопку «Сохранить веса».
- Для быстрого развертывания готовой модели нажмите кнопку «Загрузить веса». Это действие загрузит параметры из бинарного файла и принудительно переведет систему в режим эксплуатации (Epsilon сбросится до минимального значения).

4 Сообщения оператору

При взаимодействии с кнопками управления памятью нейросети (через панель администратора) могут генерироваться следующие системные сообщения:

- **«Успешно: Веса нейросети сохранены (dqn_weights.pth)!»** — Информировать об успешной сериализации текущих тензоров весовых коэффициентов в корневую папку проекта.
- **«Успешно: Обученные веса загружены! Epsilon снижен.»** — Подтверждает корректное чтение файла весов и успешный перевод комплекса в эксплуатационный режим работы.
- **«Ошибка: Модель еще не создана. Сгенерируйте хотя бы один пожар!»** — Возникает, если оператор пытается сохранить веса до инициализации структуры нейросети. Действие: необходимо дождаться генерации первого вызова в симуляции, чтобы Python-скрипт зафиксировал размерность входящего вектора состояний.
- **«Ошибка при загрузке весов...»** — Сигнализирует о физическом отсутствии файла `dqn_weights.pth` в директории или о его повреждении. Действие: проверьте наличие файла или продолжите стохастическое обучение агента «с нуля».
- **Визуальное зависание имитационной модели** — Может возникать при обрыве TCP/IP соединения между шлюзом Rureline и скриптом. Действие: перезапустите симуляцию, убедитесь, что в консоли Python нет сообщений об отсутствии библиотек `torch` или `numpy`.

[illegible]